

Classical Reinforcement Learning

*Glücklich ist, wer vergisst, was doch
nicht zu ändern ist. (Happy is he who
forgets what cannot be changed)*

Johann Strauss II

This chapter develops the algorithmic machinery that transforms the MDP foundations of Chapter 4 into practical decision-making systems. We focus on the *recursion* which motivates Bellman’s equation, with applications to dynamic programming and Q-learning, among other concepts. Other forms of model-based RL are discussed, as well as policy-gradient methods, to the extent that they are required in order to see the connections back to Causal Inference. The most notable connection discussed in this chapter is the *blip-advantage equivalence*; we will see that the advantage score used in efficient policy gradient optimization can be identical to the blip function in certain cases.

5.1 Decision Trees and Recursion

Let us return to the binary decision-making problem visualized in Figure 2.1 from Section 2.1, where a physician has to choose between two potential treatments for asthma. This diagram is a simple type of *decision tree*, where we have only two potential paths: Choose treatment 1, or choose treatment 2. Decision-making in general may require more extensive criteria. For example, if drug 1 has adverse side effects among elderly smokers, we might prescribe drug 2 for the elderly smokers, and drug 1 for everyone else. We can determine this branching decision algorithm by estimating the CATE for each subpopulation. Such policies can be represented as decision trees, where each internal node represents a test on patient characteristics, and each leaf represents a treatment assignment. The left half of Figure 5.1 illustrates this conditional treatment assignment strategy.

The programming language Python lends itself very well to the construction and usage of decision tree structures through the creation of another object class. The code to create the tree visualized in Figure 5.1 is available as follows: We create a node for each decision point, and assign downstream nodes as attributes. These downstream outcome nodes are known as the initial node’s *children*. Nodes with no children are called *terminal* nodes. For example, the node which checks the patient’s age is the parent of two children: i) a terminal node selecting drug 1, and ii) a node which checks the patient’s smoking status. Since the age node has no parents, it is called the *root* node.

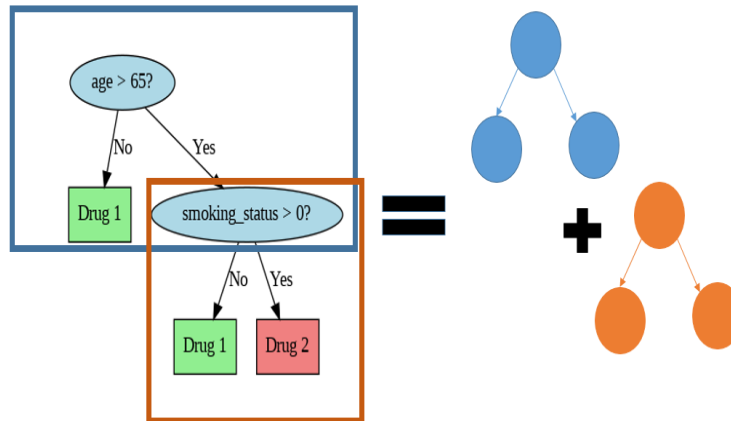


FIGURE 5.1: Visualization of the treatment decision tree. Note that the tree can be represented as the combination of two similar structures.

Python Code for Treatment Decision Tree

```
# treatment_decision_tree.py
class TreatmentNode:
    def __init__(self, feature=None, threshold=None,
                  treatment=None, node_id=None):
        self.feature = feature      # e.g., "age", "smoking_status"
        self.threshold = threshold  # e.g., 65
        self.treatment = treatment  # e.g., "drug1", "drug2"
        self.node_id = node_id     # unique name used for plotting
        self.left = None           # patients not meeting condition
        self.right = None          # patients meeting condition

    def get_label(self):
        if self.treatment: # leaf node
            return self.treatment
        return f"{self.feature} > {self.threshold}?"

    def predict_treatment(self, patient):
        if self.treatment: # leaf node
            return self.treatment

        if patient[self.feature] <= self.threshold:
            return self.left.predict_treatment(patient)
        else:
            return self.right.predict_treatment(patient)

# Build the treatment decision tree
root = TreatmentNode(feature="age", threshold=65, node_id="root")
```

```

one = TreatmentNode(treatment="Drug 1", node_id="young")
two = TreatmentNode(feature="smoking_status", threshold=0,
                    node_id="elderly_check")
three = TreatmentNode(treatment="Drug 1", node_id="elderly_nonsmoker")
four = TreatmentNode(treatment="Drug 2", node_id="elderly_smoker")
root.left = one
root.right = two
two.left = three
two.right = four

```

Computer scientists have developed an array of methods for efficient traversal of tree data structures such as the decision tree. One common approach leverages the self-similarity structure illustrated in the right half of Figure 5.1: Instead of programming a complicated set of if-else statements, we can simply define a recursive function which iterates through each level of the tree. For small trees such as the one depicted in 5.1 this is unnecessary, but in practice many more variables than age and smoking status likely play a role in determining the patient's outcome. In these more complicated cases, recursion can drastically simplify the code, collapsing what would otherwise be a sprawling set of nested if-else statements into a single self-similar function. A simple recursive function used to plot the decision tree follows.

Python Code to Create a Binary Tree

```

# treatment_decision_tree.py
from graphviz import Digraph

def visualize_treatment_tree(node, graph=None, parent=None,
                             edge_label=""):
    if graph is None:
        graph = Digraph()
        graph.attr(rankdir='TB') # Top to bottom layout

    if node:
        # Create node with descriptive label and styling
        if node.treatment: # Treatment node (leaf)
            graph.node(node.node_id,
                      label=node.get_label(),
                      shape='box',
                      style='filled',
                      fillcolor=
                        'lightgreen' if 'Drug 1' in node.treatment
                        else 'lightcoral')
        else: # Decision node
            graph.node(node.node_id,
                      label=node.get_label(),
                      shape='ellipse',
                      style='filled',
                      fillcolor='lightblue')

```

```

    # Create edge with label
    if parent:
        graph.edge(parent.node_id, node.node_id, label=edge_label)

    # Recursively visualize children
    if node.left:
        visualize_treatment_tree(node.left, graph, node, "No")
    if node.right:
        visualize_treatment_tree(node.right, graph, node, "Yes")

    return graph

```

In the sequential decision-making case represented by the Markov Decision Process, we have a decision tree situation at each timepoint. Here, the covariates (e.g. age, smoking status) are encoded into the state space. Model-based Reinforcement Learning is based on an extension of the recursion principles demonstrated here; Reinforcement Learning theory applies recursive principles *across* MDP iterations as well as *within* each timestep. This additional dimension of recursion allows us to incrementally learn the decision tree structure as the data arrive, updating our estimated optimal decision policy with each new observation.

Connection to Classical Machine Learning Methods

Decision trees have also been leveraged to create several foundational Machine Learning methods such as Monte Carlo Tree Search, which uses efficient tree traversal strategies to find the optimal path through the decision tree, i.e. the path which causes the highest estimated value of the potential reward.

Another popular tree-based method is the *random forest*. Random forests build many decision trees. Each tree in the forest is trained on a random subset of the data, using random subsets of features at each split. Instead of finding a single optimal path, random forests aggregate predictions across all trees. This ensemble approach reduces overfitting and improves generalization compared to the usage of individual trees. Random forests and related supervised ensemble methods are discussed in detail in [33].

Recursion is an extremely powerful force in mathematics and generally in life as well; indeed, it is the recursive propagation of biological phenomena from which conscious life emerges. In the words of Richard Dawkins [40], “Perhaps consciousness arises when the brain’s simulation of the world becomes so complex that it must include a model of itself.”

5.2 Dynamic Programming

A man named Richard Bellman (quoted in the epigraph of Chapter 4) was the first to use the principles of recursion to effectively derive a provably optimal policy π^* . This method, called *Dynamic Programming*, has formed the fundamental backbone of modern Reinforcement Learning. Recent advances in both computation and theory have allowed researchers to develop technology such as self-driving cars, and autonomous robots; Reinforcement Learn-

ing has also paved the way for an era of personalized medicine as well. In this chapter, we will discuss the primary methods used in this domain, building bridges from Reinforcement Learning back to our established reference, Causal Inference, developed in the previous chapters.

Bellman’s 1957 work titled “Dynamic Programming” (published first as a monograph, then synthesized in his book) is arguably the earliest piece of literature which catalyzed the development of Reinforcement Learning as a field [13]. The first major contribution of this work is the precise formulation of an optimal policy *via* the *Bellman Optimality Principle*.

Bellman was not working in isolation. In 1956, the Soviet mathematician Lev Pontryagin published what is now called the maximum principle [87, 123], an alternative approach to the same class of sequential optimization problems, using *costate variables* instead, which are entities that propagate backward through a dynamical system, weighting how perturbations at earlier times affect the final objective. The two formulations are mathematically equivalent for most problems of interest, yet Pontryagin’s work developed largely unrecognized by American researchers working on neural networks until the 80s. Nevertheless, the optimal control community that grew out of Pontryagin’s work developed extensive machinery for computing these backward-propagating quantities efficiently; Bryson and Ho’s 1969 textbook *Applied Optimal Control* [27] systematized these methods under the name “adjoint method,” providing gradient computation techniques for optimizing trajectories in aerospace, chemical engineering, and economics. Meanwhile, Bellman’s dynamic programming tradition evolved toward what we now call Reinforcement Learning, with its emphasis on value functions and policies.

It is possible - even likely - that there was some information sharing between these researchers, despite the lack of direct citations. In 1960, Bellman, Kalman, and Pontryagin all attended the First International Congress of the International Federation of Automatic Control: Bellman presented “On the application of dynamic programming to the study of control processes”, Pontryagin gave a talk titled “Optimal processes of regulation”, and Kalman presented “On the general theory of control systems” [115]. Nevertheless, for roughly three decades, the scientific communities of Kalman, Bellman, and Pontryagin developed largely in parallel, accumulating insights the other could have used. The reconnection would come in the 2010s, when deep Reinforcement Learning forced practitioners to confront both traditions simultaneously. We will return to this history in Section 5.4, where the relationship between the adjoint method and neural network training becomes explicit.

Historical Note

The 1950s and 1960s saw remarkable convergent activity on sequential decision problems across multiple fields. Bellman (dynamic programming, 1957), Pontryagin (maximum principle, 1956), and Kalman (filtering, 1960) were all developing recursive methods for optimization and estimation under uncertainty. Bellman and Kalman corresponded, and Bellman later wrote that the Kalman filter could be derived as a special case of dynamic programming [14]. The relationship between American and Soviet researchers was more complicated. Cold War constraints meant that Bellman and Pontryagin developed their approaches largely in isolation, despite working on the same class of problems (Pesch and Plail describe them as occupying opposing “corners” of an intellectual boxing match [119]). Nevertheless, Pontryagin and Bellman were aware of each other’s published work: Pontryagin presented the maximum principle at the Edinburgh ICM in 1958, while Bellman was invited as a keynote speaker at the Moscow ICM in 1966 [114].

Definition 5.1 (Bellman Optimality Principle) Recall that a policy consists of a sequence of decisions. An optimal policy has the property that “regardless of the initial state and initial decision, the remaining decisions must constitute an optimal policy with respect to the state resulting from the first decision.”[164, 13]

Definition 5.1 states the Bellman Optimality Principle formally. To give a popular analogy, suppose we want to find the shortest route from point A to point C. The optimality principle assumes that the route can be divided up into a series of segments such that if B is on the optimal route between A and C, then the optimal path from B to C is the remainder of the optimal path between A and C (Figure 5.2).

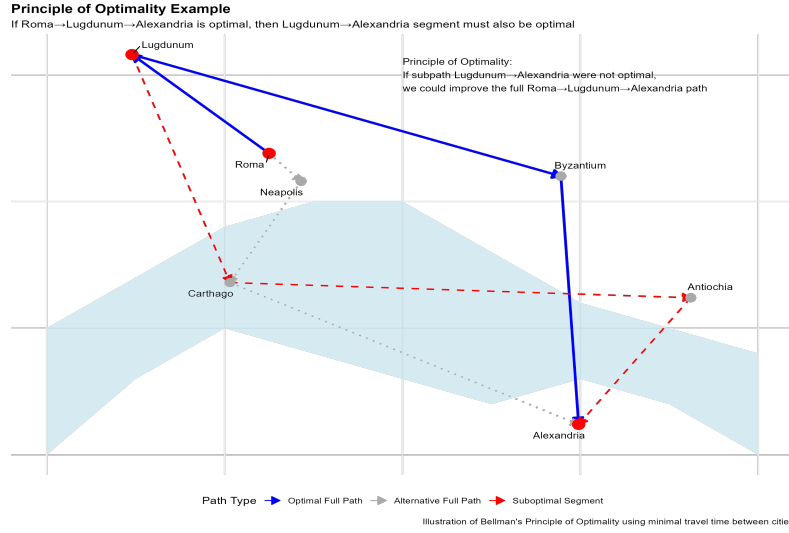


FIGURE 5.2: Illustration of the Bellman Optimality Principle.

The implicit assumption behind the Bellman Optimality Principle is that the optimal policy problem can be divided up into conditionally independent subproblems. Mathematically, this is formulated in the Markov Property, as given in definition 4.3. A counterexample arises naturally in BJJ. Suppose you have just established side control on your opponent. If you arrived at side control by passing your opponent's open guard, they have been working defensively and are likely fatigued from framing and shrimping; pressing immediately to mount might therefore be the optimal move. If, however, you arrived at the same side control by countering a failed sweep attempt, your opponent is fresher and actively attacking from below, so the optimal next move might be to consolidate your grips before advancing: Your optimal priority from side control may depend on the route that got you there, not just on the current position. So the optimality principle does not hold in this case. Note, however, that in this example the optimality principle might be recovered by incorporating a “fatigue” variable in our observable state space.

This segmentation manifests mathematically through a recursive definition of the value function. Recall the recursive average definition introduced in Equation 4.2; we can apply the same recursive estimation principle to the returns $G_j := \sum_{l=0}^{\infty} \gamma^l R_{j+l+1}$ at each step (Equation 5.1). Using this recursive formula one can thus write the value and q functions

recursively as well.

$$\begin{aligned}
 G_j &= R_{j+1} + \gamma R_{j+2} + \gamma^2 R_{j+3} + \cdots \\
 &= R_{j+1} + \gamma (R_{j+2} + \gamma R_{j+3} + \cdots) \\
 &= R_{j+1} + \gamma G_{j+1}
 \end{aligned} \tag{5.1}$$

$$\begin{aligned}
 V_\pi(x) &= \mathbb{E}_\pi [G_j \mid X_j = x] && \text{(by definition)} \\
 &= \mathbb{E}_\pi [R_{j+1} + \gamma G_{j+1} \mid X_j = x] && \text{(by 5.1)} \\
 &= \sum_{a \in \mathcal{A}} \pi(a|x) \sum_{x' \in \Omega} \Gamma(x'|x, a) [\rho(x') + \gamma \mathbb{E}_\pi [G_{j+1} \mid X_{j+1} = x']] \\
 &= \sum_{a \in \mathcal{A}} \pi(a|x) \sum_{x' \in \Omega} \Gamma(x'|x, a) [\rho(x') + \gamma V_\pi(x')]
 \end{aligned} \tag{5.2}$$

Equation 5.2 gives the recursive definition of the value function. Equation 5.2 is called the *Bellman Equation* for V_π . For continuous state spaces, the sum over $x' \in \Omega$ becomes an integral $\int_\Omega \Gamma(x'|x, a) [\cdot \cdot] dx'$.

There is also a Bellman equation for $Q(x, a)$, derived the same way. In terms of the route-finding analogy above, point A corresponds to the initial state-action pair (x_j, a_j) , and point C is the destination we wish to reach. An intermediate point B is one possible next state x_{j+1} reached after applying action a_j - so, the probability that $X_{j+1} = B$ is given by the transition function $\Gamma(B \mid x_j, a_j)$.¹

Suppose we apply the recursive definition of the value function in 5.2 to an MDP with k discrete states. Then, for a fixed policy π , we are left with a set of k equations, each characterized by $V^\pi(x)$ with a different value of the state X . One can then solve this k -dimensional system of equations using basic linear algebra, under the Markov assumption described above. This direct solution is known as *policy evaluation*. To find the *optimal* value function, we replace the policy average over actions with a maximization; the resulting system of equations is no longer linear, but it can be solved by fixed-point iteration. This iterative procedure, known as *Value Iteration*, forms the backbone of Dynamic Programming. In essence, value iteration works by iteratively improving our estimate of the optimal value function using the Bellman equation. For discrete state spaces, the value iteration update is given in 5.3.

$$\hat{V}^{(j+1)}(x) = \max_{a \in \mathcal{A}} \sum_{x' \in \Omega} \Gamma(x'|x, a) [\rho(x') + \gamma \hat{V}^{(j)}(x')] \tag{5.3}$$

For continuous state spaces, the sum becomes an integral:

$$\hat{V}^{(j+1)}(x) = \max_{a \in \mathcal{A}} \int_\Omega \Gamma(x'|x, a) [\rho(x') + \gamma \hat{V}^{(j)}(x')] dx' \tag{5.4}$$

After convergence of the value function, the optimal policy $\pi^*(x)$ given a state $x \in \Omega$ can be derived (Equation 5.5).

$$\pi^*(x) = \arg \max_{a \in \mathcal{A}} \int_\Omega \Gamma(x'|x, a) (\rho(x') + \gamma V^*(x')) dx' \tag{5.5}$$

¹In the route-finding example, the transition dynamics could be deterministic, so choosing the “go to B” action necessarily results in us traveling to B. A real-world example of this problem - along with a practical implementation of its solution - is found in A/B testing of rideshare strategies [155].

The Value Iteration algorithm is summarized in Algorithm 3.

Algorithm 3 Value Iteration

Require: MDP $(\Omega, \mathcal{A}, \Gamma, \rho)$, discount factor γ , convergence threshold ϵ

Ensure: Optimal value function V^* and optimal policy π^*

```

1: Initialize  $V_0(x)$  for all  $x \in \Omega$  (typically with zeros)
2:  $j \leftarrow 0$ 
3: repeat
4:    $\Delta \leftarrow 0$ 
5:   for each state  $x \in \Omega$  do
6:      $v \leftarrow V_j(x)$ 
7:      $V_{j+1}(x) \leftarrow \max_{a \in \mathcal{A}} \int_{\Omega} \Gamma(x'|x, a)(\rho(x') + \gamma V_j(x'))dx'$ 
8:      $\Delta \leftarrow \max(\Delta, |v - V_{j+1}(x)|)$ 
9:   end for
10:   $j \leftarrow j + 1$ 
11: until  $\Delta < \epsilon$ 
12:  $V^* \leftarrow V_j$ 
13: // Extract optimal policy
14: for each state  $x \in \Omega$  do
15:    $\pi^*(x) \leftarrow \arg \max_{a \in \mathcal{A}} \int_{\Omega} \Gamma(x'|x, a)(\rho(x') + \gamma V^*(x'))dx'$ 
16: end for
17: return  $V^*, \pi^*$ 

```

A simple Python implementation of Value Iteration follows.

Value Iteration

```
import numpy as np

def value_iteration(states, actions,
                    transition_prob, rewards, gamma=0.9, theta=1e-6):
    # Initialize value function
    V = np.zeros(len(states))

    while True:
        delta = 0
        for s in states:
            v = V[s]
            # Compute the maximum expected value over all actions
            V[s] = max(
                sum(transition_prob[s, a, s_next] *
                    (rewards[s_next] + gamma * V[s_next])
                    for s_next in states)
                for a in actions
            )
            delta = max(delta, abs(v - V[s]))

        # Check for convergence
        if delta < theta:
            break

    # Derive the optimal policy
    policy = np.zeros(len(states), dtype=int)
    for s in states:
        policy[s] = np.argmax([
            sum(transition_prob[s, a, s_next] *
                (rewards[s_next] +
                 gamma * V[s_next])
                for s_next in states)
            for a in actions
        ])

    return V, policy
```

An example usage of the above code to a simple MDP with 4 states and two actions might look like this:

```

np.random.seed(1998)

states = range(4)
actions = range(2)

transition_prob = np.random.rand(4, 2, 4)

# Reward is a function of the arrived-at state only
rewards = np.random.rand(4)

# Normalize transition probabilities
transition_prob /= transition_prob.sum(axis=2, keepdims=True)

optimal_values, optimal_policy = value_iteration(
    states, actions, transition_prob, rewards)

```

Value Iteration enjoys several convenient properties in the special case where the state transition dynamics are linear and error terms are Gaussian. Here, we do not need to iterate in order to find the optimal parameters of V . Instead, these optimal parameters can be determined algebraically using a method discovered by two people: i) Rudolf Kalman [78], to whom the method is conventionally attributed, and ii) Thorvald Thiele [92], who independently derived the same equations.

The Kalman filter, which can be derived as a special case of Dynamic Programming [14, 91], offers an online, recursive solution to state tracking. Note that, unlike in Reinforcement Learning, the Kalman filter treats the actions a_j as pre-specified inputs (not controllers that we can manipulated). However, the Kalman filter naturally extends to Reinforcement Learning applications. Kalman filtering is commonly used in spacecraft control and electrical engineering; this history and the relationship between Kalman filtering and Value Iteration is not widely known.

The convenient property of the Gaussian case is that the state transition matrix can be factored as in 5.6.

$$X_{j+1} = \Gamma_j^x X_j + \Gamma_j^a a_j + \varepsilon_j \quad (5.6)$$

The filter maintains a belief state represented by mean $\hat{X}_j$ and covariance $\hat{\Sigma}_j$. The Kalman filter equations are given in Equations 5.7- 5.11: the state and covariance predictions are computed in Equations 5.7 and 5.8 respectively, and the measurement update is implemented in Equations 5.9- 5.11.

$$\hat{X}_{j+1}^- = \Gamma_j^x \hat{X}_j + \Gamma_j^a a_j \quad (5.7)$$

$$\Sigma_{j+1}^{x-} = \Gamma_j^x \Sigma_j^x (\Gamma_j^x)^T + \Sigma_j^\varepsilon \quad (5.8)$$

In Equations 5.9-5.11, K_{j+1} represents the *Kalman Gain*, i.e. how much we trust our new observation. Here Σ_{j+1}^ν denotes the covariance of the measurement noise: In this simplified presentation, the new state X_{j+1} is observed directly, but with error. Note that K weighs the new observation by the prediction variance as a fraction of the total variance: It grows when our forecast is uncertain, and shrinks when the observation is noisy. This adjustment parallels the weighting strategy deployed in the importance sampling/IPTW

framework, which rescales the observations to account for the fact that there is higher variance associated with action-value estimates for rare actions.

$$K_{j+1} = \Sigma_{j+1}^{x-} (\Sigma_{j+1}^{x-} + \Sigma_{j+1}^{\nu})^{-1} \quad (5.9)$$

$$\hat{X}_{j+1} = \hat{X}_{j+1}^- + K_{j+1} (X_{j+1} - \hat{X}_{j+1}^-) \quad (5.10)$$

$$\Sigma_{j+1}^x = (I - K_{j+1}) \Sigma_{j+1}^{x-} \quad (5.11)$$

Equations 5.7- 5.11 represent a special case of the Kalman filtering process. The full Kalman filter also accounts for potential missingness in the state matrix X . We will return to the more generalized conceptualization later.

Recall the Expectation-Maximization Algorithm from Chapter 3, which was designed by statisticians for optimizing likelihood functions in the presence of missing data. The EM algorithm can be directly framed as a special case of value iteration, for the case where $\gamma = 0$. This connection between the EM algorithm and Value iteration is not widely known, nor - at least to the knowledge of the author at the time of writing this book - has it been explicitly proven.

Theorem 5.1 (EM Algorithm as Degenerate Value Iteration) *Let $\mathcal{M}$ be an MDP with the following structure:*

- **Actions:** *admissible parameter values $\theta \in \Theta$ for the data distribution $f(x_{obs}, X_{mis}; \theta)$, where x_{obs} is the observed part of the data and X_{mis} is the latent (missing) part of the data.*
- **State:** *the triple $x = (x_{obs}, \theta^-, \theta)$, where x_{obs} is the (fixed) observed data, $\theta \in \Theta$ is the current parameter estimate, and $\theta^- \in \Theta$ is the value taken by θ at the preceding MDP step. Note that the value of θ^- at initialization is immaterial since the greedy action never depends on θ^- .*
- **Transition:** $\Gamma((x_{obs}, \theta, \theta') \mid (x_{obs}, \theta^-, \theta), \theta') = 1$ (deterministic): *choosing θ' advances the current estimate to θ' , and the formerly current estimate θ becomes the new previous component.*
- **Reward:** $\rho((x_{obs}, \theta^-, \theta)) = \mathbb{E}_{X_{mis} \mid x_{obs}, \theta^-} [\ln f(x_{obs}, X_{mis}; \theta)] = R(\theta \mid \theta^-)$, *the reward received on arriving at the state. This is a function of the state alone ($\rho : \Omega \rightarrow \mathbb{R}$, as in Definition 4.2): the previous estimate θ^- supplies the distribution under which the missing data are imputed, and the current estimate θ is the parameter at which the complete-data log-likelihood is evaluated.*
- **Discount factor:** $\gamma = 0$.

Then value iteration on $\mathcal{M}$ produces the same sequence of iterates as the EM algorithm: From a state whose current estimate is $\theta^{(j)}$, greedy ($\gamma = 0$) action selection chooses $\theta^{(j+1)} = \arg \max_{\theta'} \rho((x_{obs}, \theta^{(j)}, \theta')) = \arg \max_{\theta'} R(\theta' \mid \theta^{(j)})$, which is the M-step; the reward then collected on arrival, $R(\theta^{(j+1)} \mid \theta^{(j)})$, is the E-step objective.

Proof

Consider an MDP with the specifications in Theorem 5.1 (recall that the data component x_{obs} is fixed, so the state evolves only through its parameter components (θ^-, θ)).

With $\gamma = 0$, the Bellman optimality equation built from Equation 5.2 retains only the reward-on-arrival term. Under the deterministic transition above, the state $(x_{obs}, \theta^-, \theta)$ has optimal value

$$V^*((x_{obs}, \theta^-, \theta)) = \max_{\theta'} \rho((x_{obs}, \theta, \theta')) = \max_{\theta'} R(\theta' \mid \theta),$$

since the action θ' leads to the arrived-at state $(x_{\text{obs}}, \theta, \theta')$, whose reward is $R(\theta' | \theta)$. The maximizer depends on the current estimate θ but not on θ^- .

Value iteration therefore reduces to greedy action selection: At iteration j , with current estimate $\theta^{(j)}$, the value iteration update gives us Equation 5.12.

$$\theta^{(j+1)} = \arg \max_{\theta'} R(\theta' | \theta^{(j)}) = \arg \max_{\theta'} \mathbb{E}_{X_{\text{mis}} | x_{\text{obs}}, \theta^{(j)}} [\ln f(x_{\text{obs}}, X_{\text{mis}}; \theta')]. \quad (5.12)$$

But Equation 5.12 is the M-step of EM, and the reward $R(\theta' | \theta^{(j)})$ being maximized is the E-step objective, i.e. the expected complete-data log-likelihood under the imputation distribution fixed by the current estimate $\theta^{(j)}$. The reward realized on arriving at the next state is $\rho((x_{\text{obs}}, \theta^{(j)}, \theta^{(j+1)})) = R(\theta^{(j+1)} | \theta^{(j)})$, which is the EM objective value at step j .

Theorem 5.1 naturally arises when one understands the maximum likelihood principle as a problem of optimal control, following the principles described in [143].

TABLE 5.1: Notation mapping between the EM algorithm and value iteration, as used in Theorem 5.1.

EM Algorithm	Value Iteration	Role
Candidate parameter θ'	Action a	Decision variable
Triple $(x_{\text{obs}}, \theta^-, \theta)$	State x	Current context
E-step: evaluate $\rho((x_{\text{obs}}, \theta^-, \theta)) = R(\theta \theta^-)$	Reward $\rho(x')$	Reward evaluation
M-step: $\arg \max_{\theta'} \rho$	Greedy action: $\arg \max_a Q(x, a)$	Policy improvement
Log-likelihood $\ln f$	Reward function ρ	Objective
$\gamma = 0$ (implicit)	$\gamma = 0$ (explicit)	No future consequences

What happens when we allow $\gamma > 0$ in the EM version of value iteration? This extended EM algorithm would have us choosing parameters to maximize Equation 5.14. In terms of the augmented MDP of Theorem 5.1, allowing $\gamma > 0$ adds the discounted value-to-go to the immediate reward. Because the observed data are fixed and the value-to-go from a state depends only on its current estimate, and *not* on the previous component θ^- , we write this value as $V(\theta)$. Similarly, we write the immediate reward of advancing the current estimate from $\theta^{(j)}$ to a candidate θ' as $R(\theta' | \theta^{(j)})$.

Recall that in standard EM, we define the expected complete-data log-likelihood by integrating over all possible values of the missing data X_{mis} (Equation 5.13).

$$R(\theta' | \theta) = \int \ln f(x_{\text{obs}}, X_{\text{mis}}; \theta') \cdot f(X_{\text{mis}} | x_{\text{obs}}, \theta) dX_{\text{mis}} \quad (5.13)$$

Here, θ is our current parameter estimate, which defines the conditional distribution $f(X_{\text{mis}} | x_{\text{obs}}, \theta)$ used for integration, and θ' is the candidate parameter being evaluated in the complete-data log-likelihood $\ln f(x_{\text{obs}}, X_{\text{mis}}; \theta')$. The integration is performed across all possible values of the unobserved (missing) data.

In the EM literature, the expected complete-data log-likelihood is conventionally denoted $Q(\theta' | \theta)$. To avoid confusion with the action-value function $Q(x, a)$ from Reinforcement Learning, we instead use $R(\theta' | \theta)$ throughout this section.

We can thus derive the lookahead EM objective as Equation 5.14, where $R(\theta' | \theta^{(j)})$ is the immediate expected reward from choosing candidate θ' , and $\hat{V}(\theta'; \theta^{(j)})$ approximates

the future value of being at parameter θ' , with the approximation anchored at the current parameter $\theta^{(j)}$.

$$\theta^{(j+1)} = \arg \max_{\theta'} \left[R(\theta' | \theta^{(j)}) + \gamma \cdot \hat{V}(\theta'; \theta^{(j)}) \right] \quad (5.14)$$

To make this problem tractable, we need to approximate $\hat{V}(\theta; \theta^{(j)})$. The value function represents the best achievable reward from a given parameter (see Equation 5.15).

$$V(\theta) = \max_{\theta'} R(\theta' | \theta) = \max_{\theta'} \int \ln f(x_{\text{obs}}, X_{\text{mis}}; \theta') \cdot f(X_{\text{mis}} | x_{\text{obs}}, \theta) dX_{\text{mis}} \quad (5.15)$$

We approximate this using a second-order Taylor expansion around $\theta^{(j)}$ (Equation 5.16), where $\mathbf{H}$ is the Hessian matrix.

$$\begin{aligned} R(\theta' | \theta) &\approx R(\theta | \theta) + \nabla_{\theta'} R(\theta' | \theta) \big|_{\theta'=\theta} \cdot (\theta' - \theta) \\ &\quad + \frac{1}{2} (\theta' - \theta)^T \mathbf{H} (\theta' - \theta) \end{aligned} \quad (5.16)$$

The maximum occurs at

$$\theta'^* \approx \theta - \mathbf{H}^{-1} \nabla_{\theta'} R(\theta' | \theta) \big|_{\theta'=\theta} \quad (5.17)$$

and thus

$$V(\theta) \approx R(\theta | \theta) - \frac{1}{2} \nabla R^T \mathbf{H}^{-1} \nabla R, \quad (5.18)$$

where ∇R is a vector of gradients $\{\frac{\partial R}{\partial \theta_i}\}$ of the same length as the parameter vector θ , and where $\nabla R, \mathbf{H}$ are evaluated at $\theta^{(j)}$.

∇R plays an analogous role to the Kalman gain from Equations 5.7- 5.11, as well as to the propensity score weights from section 3.3. All of these elements are weighting strategies that adjust an estimate to place more emphasis on the more *informative* observations, upweighting the “surprising” observations with high Fisher information. Through these adjustment factors, we update our model more when we observe something with a large sampling space: Observing $X = x$ when X could not have been otherwise is uninformative, but observing $X = x$ when $\text{Var}(X)$ is high is very informative. Another way to say this is that if a variable X is more informative if there are a lot of plausible counterfactuals for X .

With this gradient-based approximation of $V(\theta)$ in hand, one may then define the lookahead EM step according to Equation 5.19.

$$\theta^{j+1} = \arg \max_{\theta'} \left(R(\theta' | \theta^{(j)}) + \gamma \left[R(\theta^{(j)} | \theta^{(j)}) - \frac{1}{2} \nabla_{\theta'} R^T \mathbf{H}^{-1} \nabla_{\theta'} R \right] \right) \quad (5.19)$$

Optimizing this objective using the second-order gradient and Hessian is also the strategy employed by Newton’s method²; in fact, lookahead EM can be viewed as a quasi-Newton method similar to the methods proposed in [71, 90].

²I assume most readers are familiar with Newton’s method, which is taught in most undergraduate Calculus classes; if not, an overview can be found in [122].

Unlike genuine Reinforcement Learning problems, in Equation 5.14 there are no state transitions in the usual sense. The data $\mathbf{x}$ is fixed: The “state space” is merely our current position in a fixed optimization landscape, and we have complete control over our trajectory. Our problem is therefore reduced to simple optimization.

Within the local region where Newton’s method converges (where the first two terms of the Taylor series expansion suffice for approximation), the lookahead EM method should provide meaningful improvement over standard EM’s first-order approach for problems with convex loss landscapes. Indeed, recent research has confirmed that EM can be accelerated by incorporating curvature information [71, 90]. These methods rest on the assumption that near a fixed point θ^* , the EM update map satisfies Equation 5.20, where $\text{EM}(\theta) = \text{argmax}_{\theta'} R(\theta'|\theta) - \theta^*$ is the recentered EM update and J is its Jacobian evaluated at θ^* .

$$\text{EM}(\theta) \approx J(\theta - \theta^*) \quad (5.20)$$

The rate of convergence of the EM algorithm is governed by the *fraction of missing information*, i.e. the proportion of Fisher information in the complete data that is lost due to missingness [41, 105]. This rate of convergence is given by the *spectral radius* of J , which we denote $\xi(J) \in (0, 1)$. When $\xi(J)$ is close to 1 (high missing information), EM converges slowly, motivating acceleration methods that incorporate curvature information.

In the EM literature, the spectral radius is conventionally denoted ρ ; we use ξ instead to avoid dual usage of ρ , which denotes the reward function in our Reinforcement Learning parlance.

SQUAREM is a state-of-the-art modern optimization method based on these quadratic quasi-Newton methods for extended EM optimization [43]. One might expect the lookahead EM algorithm proposed in Equation 5.19 to improve optimization over SQUAREM for applications where the loss landscape is well-behaved, though it will be substantially slower due to expensive gradient computation. Lookahead EM may also fail in cases where the loss landscape is not convex, in which case the estimation of second-order derivatives is unhelpful.

A summary of some control methods we have discussed in this section, classified according to state/action configuration, is shown in Table 5.2.

TABLE 5.2: Classification of optimization and control methods by state and action dynamics.

	Static Action	Dynamic Action
Static State	Closed-form optimization	Newton’s method / EM algorithm
Dynamic State	Kalman filtering	Value iteration / Dynamic programming

5.3 Q-learning

While the value function $V_\pi(x)$ estimates the expected return starting from state x and following policy π , it does not directly tell us which action to take. Instead, we have to derive

the optimal policy separately, iterating between policy evaluation and value estimation in Algorithm 3. In 1989, Chris Watkins developed a method called *Q-learning* to address this issue; this seminal Q-learning paper [178] is foundational upon the cornerstone textbook *Applied Dynamic Programming* by Bellman & Dreyfus [15], who worked together at RAND. Interestingly, the seminal Q-learning paper [178] also includes a citation to an early book on heuristic search strategies by Judea Pearl [116], who later became one of the early founding researchers of Causal Inference, and whose 2018 book *The Book of Why* [118] remains one of the most popular cornerstones of Causal Inference to this day.

Instead of dealing with value functions and policy estimation separately, Q-learning learns the action-value function $Q(x, a)$ directly. The Q function represents the expected return starting from state x , taking action a , and thereafter following an optimal policy.

The Q-learning update rule is given by Equation 5.21; here, $\alpha \in (0, 1)$ is the learning rate and γ is the discount factor. Higher values of α induce faster learning, at the potential cost of instability (if the step size is too large, the estimates may fail to converge).

$$Q(x_j, a_j) \leftarrow Q(x_j, a_j) + \alpha[R_{j+1} + \gamma \max_{a'} Q(x_{j+1}, a') - Q(x_j, a_j)] \quad (5.21)$$

Once we have an estimate of the Q -function, deriving a policy is straightforward: At each state, select the action with the highest estimated Q -value, per equation 5.22.

$$\pi(x) = \arg \max_{a \in \mathcal{A}} \hat{Q}(x, a) \quad (5.22)$$

Note that Equation 5.21 directly approximates the optimal action-value function Q^* regardless of the policy being followed: The system's prescribed policy is different from the behavior policy. This is called *off-policy* behavior, and it is analogous to the situation in Causal Inference where randomization is not possible.

To allow for sufficient exploration in the early sampling stages, Q-learning implementations typically employ some strategy such as ϵ -greedy action selection during learning.

In the clinical trials example where the state X contains patient covariates, $Q(x, A = \text{Treatment}) - Q(x, A = \text{Placebo})$ is the conditional average treatment effect. Integrating over the marginal distribution of states gives us the average treatment effect:

$$\text{ATE} = \int_{\Omega} [Q(x, a = \text{Treatment}) - Q(x, a = \text{Placebo})] p(x) dx \quad (5.23)$$

where $p(x)$ is the marginal distribution of states in the population. In practice, $p(x)$ can be estimated from the empirical distribution of patient covariates in the study, or it may represent the stationary distribution $\mu^\pi(x)$ of a state x under a specific policy π .

To illustrate the practical implementation of tabular Q-learning, we present GrappleBot, a Reinforcement Learning agent that learns basic BJJ standup strategy through self-play against a stochastic opponent. This example demonstrates how to construct discrete state spaces, implement Q -tables, and apply the Q-learning update rule in a structured domain.

The GrappleBot environment models tactical interactions in the standup part of BJJ³ (where the goal is to throw your opponent to the ground) as a discrete-time MDP. The state space Ω is constructed as the Cartesian product of the following variables:

³Martial arts practitioners may recognize this framework as Judo (more or less), which was derived from Japanese Jujutsu by Jigoro Kano; later, Kano's students invented Brazilian Jiu Jitsu.

- Self grip status: $\{0, 1\}^2$ (left grip, right grip)
- Opponent grip status: $\{0, 1\}^2$ (left grip, right grip)
- Self stance: $\{0, 1, 2\}$ (right-forward, square, left-forward)
- Opponent stance: $\{0, 1, 2\}$ (right-forward, square, left-forward)
- Self fatigue: $\{0, 1\}$ (fresh, fatigued)
- Opponent fatigue: $\{0, 1\}$ (fresh, fatigued)
- Game outcome: $\{-1, 0, 1\}$ (loss, ongoing, win)

This yields a total state space size of $|\Omega| = 2^2 \times 2^2 \times 3 \times 3 \times 2 \times 2 \times 3 = 1728$ states.

The action space $\mathcal{A}$ consists of seven tactical actions:

$$\mathcal{A} = \{\text{gripL}, \text{gripR}, \text{breakgripL}, \text{breakgripR}, \text{mv}, \text{throwEW}, \text{throwNS}\}$$

where “throwEW” represents east-west throws (resulting in side falls) and “throwNS” represents north-south throws (resulting in back falls).

The reward function $\rho : \Omega \rightarrow \mathbb{R}$ implements sparse rewards:

$$\rho(x_{j+1}) = \begin{cases} +1 & \text{if agent wins} \\ -1 & \text{if agent loses} \\ 0 & \text{otherwise} \end{cases}$$

Loss occurs if the opponent successfully throws the agent or if the episode times out after $T = 10$ timesteps without resolution.

Throw success probabilities are determined by a lookup table conditioned on stance, fatigue, and grip configuration. For the agent’s throws, success probability $p \in \{0.1, 0.5, 0.9\}$ corresponding to low, medium, and high success rates based on tactical advantage. The opponent’s throws succeed deterministically when holding both grips, representing a simplified model of superior technique.

The environment implements sequential decision-making where the agent acts first, followed by the opponent. State transitions $\Gamma(x_{j+1}|x_j, a_j)$ incorporate both the agent’s action and a subsequent random opponent action drawn from the set of valid moves in the resulting state. Pseudocode for creation of the GrappleBot Environment class is available in Algorithm 4; here `MapProb` is a dictionary mapping the qualitative labels (low, medium, high) to probability values, and `ComputeLabel` is a helper function defined in Algorithm 5.

Fatigue mechanics introduce temporal dependencies: stamina decreases by 0.5 per action and by 4 for failed throws. When stamina drops below 15, the player becomes fatigued, reducing throw effectiveness.

The agent maintains a Q-table $Q : \Omega \times \mathcal{A} \rightarrow \mathbb{R}$ initialized to zeros. Action selection follows an ϵ -greedy policy.

$$\pi(a_j|x_j) = \begin{cases} \frac{\epsilon}{|\mathcal{A}_{\text{valid}}|} & \text{if } a_j \text{ is valid but not greedy} \\ 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}_{\text{valid}}|} & \text{if } a_j = \arg \max_{a \in \mathcal{A}_{\text{valid}}} Q(x_j, a) \\ 0 & \text{if } a_j \text{ is invalid} \end{cases}$$

where $\mathcal{A}_{\text{valid}} \subseteq \mathcal{A}$ represents actions available in state x_j . For example, grip-breaking actions are only valid when the opponent holds the corresponding grip.

Algorithm 4 GrappleBot Environment Initialization

Require: None**Ensure:** Initialized environment with state space Ω , action space $\mathcal{A}$, throw probability table T , state indexing mappingsDefine grip indicators: $G_L^S, G_R^S, G_L^O, G_R^O \in \{0, 1\}$ Define stances: $S^S, S^O \in \{0, 1, 2\}$ ▷ 0: right, 1: square, 2: leftDefine fatigue: $F^S, F^O \in \{0, 1\}$ Define match outcome: $W \in \{-1, 0, 1\}$ ▷ -1: loss, 0: ongoing, 1: win $\Omega \leftarrow$ Cartesian product of all state components $|\Omega| \leftarrow 2^4 \times 3^2 \times 2^2 \times 3 = 1728$ $x_0 \leftarrow (0, 0, 0, 0, 0, 0, 0, 0, 0)$ ▷ Initial state: no grips, right-forward stances, no fatiguestamina_S $\leftarrow$ 20, stamina_O $\leftarrow$ 20max_steps $\leftarrow$ 10 $\mathcal{A} \leftarrow \{\text{gripL, gripR, breakgripL, breakgripR, mv, throwEW, throwNS}\}$

▷ Build throw success probability table

Initialize empty table T **for** each $s^S \in \{0, 1, 2\}$ **do** **for** each $f^S \in \{0, 1\}$ **do** **for** each $f^O \in \{0, 1\}$ **do** **for** each $g \in \{\text{None, Left, Right, Both}\}$ **do** **for** each $\theta \in \{\text{throwNS, throwEW}\}$ **do** $L \leftarrow \text{ComputeLabel}(s^S, f^S, f^O, g, \theta)$ $T[(s^S, f^S, f^O, g, \theta)] \leftarrow \text{MapProb}(L)$ **end for** **end for** **end for** **end for****end for**

▷ Build state indexing mappings

 $\mathcal{T} \leftarrow$ enumerate all tuples in Ω lexicographically**for** $i \leftarrow 0$ to $|\Omega| - 1$ **do** StateKey[i] $\leftarrow \mathcal{T}[i]$ TupleToIdx[$\mathcal{T}[i]$] $\leftarrow i$ **end for****return** $\Omega, \mathcal{A}, T, \text{StateKey}, \text{TupleToIdx}$

Algorithm 5 ComputeLabel Helper Function**Require:** Stance s^S , self fatigue f^S , opponent fatigue f^O , grips g , throw type θ **Ensure:** Qualitative label $L \in \{\text{Low, Medium, High}\}$

```

both_grip  $\leftarrow (g = \text{Both})$ 
any_grip  $\leftarrow (g \in \{\text{Left, Right, Both}\})$ 
if both_grip or ( $f^O = 1$  and  $f^S = 0$ ) then
    return High
end if
if  $f^O = 1$  or  $f^S = 0$  then
    if  $\theta = \text{throwNS}$  and  $s^S = 1$  and any_grip then
        return Medium
    end if
    if  $\theta = \text{throwEW}$  and  $s^S = 0$  and  $g = \text{Right}$  then
        return Medium
    end if
end if
return Low

```

The Q-learning update rule follows Equation 5.21 with learning rate $\alpha = 0.5$, discount factor $\gamma = 0.99$, and exploration rate ϵ decaying from 1.0 to 0.1 according to $\epsilon_{j+1} = \max(0.1, 0.9995 \cdot \epsilon_j)$.

Figure 5.3 shows the learning curve over 3000 training episodes followed by 1000 test episodes. The agent achieves convergence to a win rate of approximately 70% against the random opponent, demonstrating that GrappleBot successfully learned the optimal policy of acquiring grips while preventing the opponent from gripping, then throwing once the grips are on. The GrappleBot code is available in Appendix D.2.

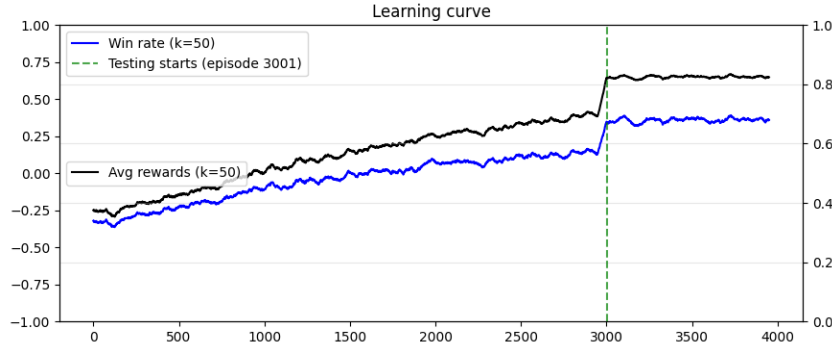


FIGURE 5.3: GrappleBot learning curve showing average reward (black) and win rate (blue) over episodes. The vertical green line indicates the transition from training to testing.

One potential benefit of Q -learning over Dynamic Programming is that it relaxes the assumed structure on the state transition matrix Γ : It is considered to be a *non-parametric* method. Hence, Q -learning tends to find more general solutions than Dynamic Programming finds on large sample sizes. However, this increased generalizability is purchased at the cost of efficiency, since the additional flexibility of the model means more parameters to fit. In most practical applications, the state space Ω is too large to represent the Q -function as a table.

Q -learning is considered “non-parametric”, but in reality, no method is *completely* non-

parametric - there are always model assumptions. The structure of the Q function is the critical assumption in Q -learning: At the end of the day, the Q -function needs to know what to do with the state, action, and some fittable parameters in order to return some informative value. In practice, the appropriate structure for Q is different for every application; this structure can be found by balancing efficiency and generalizability for the domain-specific problem at hand.

5.4 Deep Q-learning

The tabular Q -learning method introduced in the previous section is intractable in many practical applications, due to the complexity of the system that must be approximated by $Q(x, a)$. In 2015, a method which leverages Deep Neural Networks to approximate Q was developed to handle this case; this method, known as *Deep Q-learning* [108], is still widely used today in modern applications of Reinforcement Learning. The dawn of deep Q -learning is what popularized Reinforcement Learning as a powerful AI technique; among other applications, deep Q -learning has been used to develop autonomous robots and self-driving cars.

Recall the complex DAG, depicting a neural network, which was pictured in Figure 3.7. Each node in the neural network represents a mathematical operation that transforms its inputs using some parameters θ ; for example they could be simple linear regression models which take x and return θx (though most of the time slightly more complicated functions such as the sigmoid are typically preferred; refer to any introductory tutorial on deep neural networks for details). If the neural network has many layers, we say it is *deep*. If it has many nodes in each layer, we say it is *wide*. The *acyclic* structure of neural networks enables an efficient parameter estimation method called *backpropagation*. The backpropagation algorithm used to train neural networks was first discovered by Dreyfus, the control theorist who worked with Bellman to write the book *Applied Dynamic Programming* which as mentioned in section 5.3 was cited in the original Q -learning paper. Backpropagation was later adopted by the neural network community, in the research lineage descending from Rosenblatt's perceptron, to train neural networks (see p.11 of [173]).

Suppose we want to minimize some loss function $\mathcal{L}(\theta)$ that measures how well our network approximates the true Q -function. To update parameters using gradient descent, we need $\frac{\partial \mathcal{L}}{\partial \theta_i}$ for each parameter θ_i . In a network with L layers, the chain rule gives us Equation 5.24; this process is called *backpropagation* of errors. Here, each term in the chain acts as an information weight that determines how much the error signal should influence earlier layers.

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = \frac{\partial \mathcal{L}}{\partial h_L} \cdot \frac{\partial h_L}{\partial h_{L-1}} \cdot \dots \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial \theta_1} \quad (5.24)$$

Note that each Jacobian term $\frac{\partial h_\ell}{\partial h_{\ell-1}}$ weights how much the error signal from layer ℓ should propagate to layer $\ell - 1$. When this Jacobian is large, layer ℓ is highly sensitive to changes in layer $\ell - 1$, so information flows quickly, like a fluid with low viscosity such as water. When the gradient is small, the layers are nearly decoupled, and the gradient signal attenuates, flowing more slowly like a fluid with higher viscosity such as honey. The neural network's layers, in the physics analogy, correspond to physical units in space. Moreover, the denominator in each term of the chained weighting structure in 5.24 is analogous to the other weighting schemes we have discussed, serving the same function as the IPT weights

in MSMs, gradient factors in the quasi-Newton methods, and the Kalman gain in Kalman filtering.

The neural network’s acyclic structure guarantees that our chain of partial derivatives is well-defined. Since the series of transformations is sequential, we can compute gradients by moving backwards through the network from output to input, with each layer depending only on its predecessors. If the network contained cycles, this backward pass would be undefined as a single sequential sweep (recurrent networks work around this by unrolling the cycle in time); we would need to solve for gradients simultaneously rather than sequentially, since the gradient of each node might depend on the gradient of other nodes. Consider the fluid analogy: Suppose water flows along a channel into a sink. Once the source of the water is turned off, then there is a finite time before all of the water drains into the sink. In the DNN example, adding a cup of water to the source in this system corresponds to feeding a set of samples into the system; the information from these samples flows through the network via backpropagation until it reaches the end of the network (sink) as in Figure 5.4.

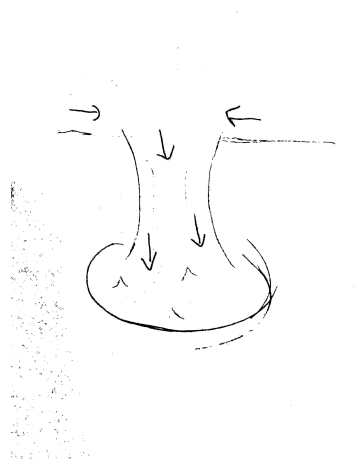


FIGURE 5.4: Diagram of water flowing unidirectionally into a sink. Arrows are analogous to gradients, and the bottom of the waterfall is the “sink” analogous to the head of the neural network.

But consider the situation where instead of running into a sink, the water in your system flows in a circle (Figure 5.5). Then, the cycle may go on forever. This is what would happen to the information in the gradient chain if our network was *cyclic* instead of *acyclic*: a single backward pass of gradient descent would have no well-defined termination criterion, as gradients would have circular dependencies.

The *feed-forward* (acyclic) structure of DNNs is analogous to *acyclicity* in causal DAGs; this analogy naturally arises if we interpret a neural network as an abstract DAG for a problem with unknown causal structure. In a DAG, acyclicity allows separation of nodes, which subsequently allows us to identify complex effects by sequential estimation. We can update each parameter of the DAG in a well-defined order because causality flows in one direction: If there were some feedback loop in the system, then the causal system would be intractable.

The backward pass we have just described has a history that predates neural networks by decades. Recall from Section 5.2 that Pontryagin’s maximum principle provided an alternative to Bellman’s dynamic programming for sequential optimization. Moreover, in this

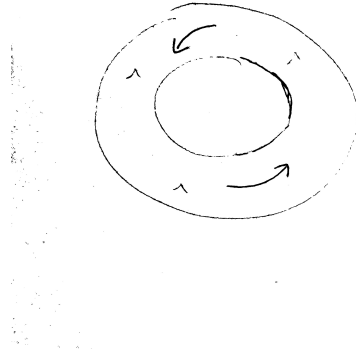


FIGURE 5.5: Diagram of water flowing circularly. Arrows represent gradients; the momentum of the water within each arrow corresponds to the magnitude of the gradient.

work, Pontryagin introduced machinery for optimal control that turns out to be mathematically identical to backpropagation [93]. To see this, we begin with Pontryagin’s *costate variables*.

Consider a discrete-time system where we wish to minimize a total cost $J = \sum_{j=0}^T L_j(x_j, a_j)$ subject to dynamics $x_{j+1} = f_j(x_j, a_j)$. Here L_j is a per-step cost (the negative of our reward ρ), and f_j encodes the state transitions. The relevant question is: *how does the total cost J change if we perturb our action a_j at some intermediate step?*

I will focus, here, on the deterministic case $x_{j+1} = f_j(x_j, a_j)$ rather than the stochastic MDP transition $x_{j+1} \sim \Gamma(\cdot|x_j, a_j)$ because the gradient computation is cleaner when we do not have to worry about differentiating through a probability distribution. The idea is the same in the stochastic setting, though one must either differentiate through the expected dynamics $\mathbb{E}[x_{j+1}|x_j, a_j]$, or use the “reparameterization trick” to express stochastic transitions as deterministic functions of noise variables: $x_{j+1} = f_j(x_j, a_j, \varepsilon_j)$ where $\varepsilon_j \sim \mathcal{N}(0, I)$. The latter approach, now standard in variational autoencoders and stochastic policy gradients, allows backpropagation through stochastic computational graphs. For our purposes, the deterministic presentation suffices to establish the structural parallel with neural network training.

The costate variable λ_j is defined as the sensitivity of the optimal cost-to-go to perturbations in the state at step j (Equation 5.25), where $J_{j \rightarrow T}^*$ denotes the minimum achievable cost from step j onward.

$$\lambda_j = \frac{\partial J_{j \rightarrow T}^*}{\partial x_j} \quad (5.25)$$

The costate tells us how much we should “care” about being in a particular state: If λ_j is large, small changes in x_j propagate forward to create large changes in the final objective.

Applying the chain rule gives us the backwards recurrence relation on λ_j expressed in 5.26; this relation is known as the *adjoint equation*.

$$\lambda_j = \frac{\partial L_j}{\partial x_j} + \left(\frac{\partial f_j}{\partial x_j} \right)^\top \lambda_{j+1} \quad (5.26)$$

Starting from a terminal condition $\lambda_T = \frac{\partial L_T}{\partial x_T}$, we propagate backward through the system, accumulating at each step the direct effect on the local cost ($\frac{\partial L_j}{\partial x_j}$) plus the indirect effect transmitted through the dynamics ($(\frac{\partial f_j}{\partial x_j})^\top \lambda_{j+1}$).

Now compare 5.26 to backpropagation in a neural network. Let h_j denote the hidden state at layer j , with $h_{j+1} = \sigma(W_j h_j)$ for weights W_j and activation function σ . Defining $\delta_j = \frac{\partial \mathcal{L}}{\partial h_j}$ as the gradient of the loss with respect to the hidden state, we can obtain Equation 5.27.

$$\delta_j = \left(\frac{\partial h_{j+1}}{\partial h_j} \right)^\top \delta_{j+1} = W_j^\top \text{diag}(\sigma'(W_j h_j)) \delta_{j+1} \quad (5.27)$$

Equations 5.26 and 5.27 characterize the same recurrence relation. The costate λ_j and the error signal δ_j each serve the same purpose within their respective equations, representing the sensitivity of a downstream objective to perturbations at an intermediate stage. These sensitivities are then propagated backward via the transpose of the forward dynamics' Jacobian, differing only in their notation and the specific form of the functions involved.

Bryson and Ho derived Equation 5.26 in 1969 for aerospace trajectory optimization. Some five years later, Werbos recognized in 1974 that the same machinery could train neural networks [181]. When Rumelhart, Hinton, and Williams popularized the technique in 1986, they called it “backpropagation of errors” and the optimal control lineage faded from the machine learning community’s awareness [140]. For thirty years, these two communities developed increasingly sophisticated versions of the same algorithm in near-isolation.

In optimal control, once we have computed the costates λ_j , the gradient of the total cost with respect to the control a_j can be expressed as in Equation 5.28.

$$\frac{\partial J}{\partial a_j} = \frac{\partial L_j}{\partial a_j} + \lambda_{j+1}^\top \frac{\partial f_j}{\partial a_j}. \quad (5.28)$$

But this is just the gradient of a neural network with respect to the weights; Equation 5.28 is exactly the direct effect on the local loss plus the downstream effect mediated by the error signal. As previously mentioned, the Kalman gain from Section 5.2, the IPT weights from Chapter 2, and the costate variables here all serve the same function, weighting updates according to how informative each observation or perturbation is. High sensitivity means the observation carries substantial information about the objective; low sensitivity means it can be safely downweighted. This principle - that not all data points contribute equally to learning, and that optimal estimation requires tracking these contributions - recurs throughout statistical and control-theoretic approaches to sequential problems. Other examples that we will not discuss in detail include attention mechanisms [174], market cap weights in finance [124], and unit conversions in classical physics [54].

The deep Reinforcement Learning revolution reunited the fields of deep learning and control theory. When we train a neural network to approximate $Q(x, a; \theta)$ by minimizing the loss in Equation 5.29, we are using Bryson and Ho’s adjoint method to optimize a parametric approximation to Bellman’s value function.

“Deepening” the Q-learning algorithm revolutionized the field of optimal control because of how general deep neural networks can be; this generality arises from their capability to support the incorporation of big data to fit a large number of parameters. However, even within the class of neural networks, some models might be more general than others: we are once again faced with the pervasive struggle between formalism (structure) and empiricism (flexibility) mentioned in our discussion of the tradeoffs between parametric statistical models (which are more formalist) and deep neural networks (which are more flexible). Within the class of neural networks, models which are very “wide” tend to memorize the data, performing well on data which closely resemble the training data, but more poorly

on out-of-sample distributions. Conversely, “deep” neural networks extract abstract forms. These models, when they converge, tend to yield more general results. The caveat is that convergence is not guaranteed for deep Q-learning, and is not, in fact, guaranteed generally for wide networks either.

The problem is that any time we use a neural network to approximate Q , we are creating a combination of elements known as the *deadly triad*: Using (i) function approximation, (ii) off-policy learning, and (iii) bootstrapping all in the same system is notoriously dangerous (see [164] for further discussion). Wide networks experience this instability less severely because memorization limits how far errors can propagate through the bootstrapped estimates, but the problem can still cause compounding errors that prevent convergence. Deep networks, with their greater reliance on abstraction and shared parameters, are subject to the full force of the deadly triad’s instability.

Convergence might also be hindered by a variety of factors not limited to sample size. For example, the parametrization and boundedness of the reward function is extremely important. Like the episodic versus continuing formulations in Section 4.3, the choice between wide and deep architectures involves tradeoffs between interpretability and generalization.

Let $Q(x, a; \theta)$ represent the Q -function approximated by a neural network with parameters θ . The network takes as input the state x and outputs Q -values for each possible action in $\mathcal{A}$. The parameters θ are updated to minimize the loss function $\mathcal{L}$, which is defined in Equation 5.29, where θ^- represents the parameters of a target network that is periodically updated to match θ .

$$\mathcal{L}(\theta) = \mathbb{E}[(R_{j+1} + \gamma \max_{a'} Q(x_{j+1}, a'; \theta^-) - Q(x_j, a_j; \theta))^2] \quad (5.29)$$

There are two Python packages commonly used to implement Deep Neural Networks: `keras` and `PyTorch`. `keras` is generally what students use to start with; it is more user-friendly and lends itself better to integration with functional programming strategies. However, `PyTorch` is more flexible than `keras`; it is easier to design custom workflows and training loops in `PyTorch` than `keras`. Consequently, `PyTorch` is typically preferred by researchers, and developers producing complex neural architectures. A simple `PyTorch` implementation of a Neural Network class follows.

Deep Q Network Initialization using PyTorch

```
#load PyTorch
import torch
import torch.nn as nn
# Simple DQN
class DQN(nn.Module):
    def __init__(self, input_size, output_size):
        super(DQN, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(input_size, 64),
            nn.ReLU(),
            nn.Linear(64, output_size)
        )

    def forward(self, x):
        return self.fc(x)
```

Here, the `super(DQN, self).__init__()` line invokes the initializer of the parent `nn.Module` class from `PyTorch`, setting up the standard module machinery before our

own layers are defined. Upon initialization, the defined `DQN` class will initialize a fully connected one-layer neural network `fc`, with 64 nodes in the fully connected layer. The function `forward` pushes the input data `x` through the defined neural architecture.

Practical applications of Deep Q-learning almost always incorporate a technique called *experience replay*, where state-action-reward-next state tuples $(x_j, a_j, R_{j+1}, x_{j+1})$ are stored in a replay buffer. Training is performed by randomly sampling batches from this buffer, which breaks the correlation between consecutive samples and stabilizes learning. A function which implements Deep Q-learning with experience replay follows, assuming you have an environment object `env` which has attributes corresponding to the observation and action space.

Simple Deep Q Learning Training Loop

```
from collections import deque
import torch.optim as optim
import torch.nn.functional as F
from tqdm import trange

# Training loop
def train_dqn(env, episodes=2000, seed=42,
             gamma=0.95, batch_size=64,
             epsilon_min = 0.2, epsilon_decay=0.99995, lr=0.0001):

    model = DQN(input_size=env.obs_space, output_size=env.action_space)
    target_model = DQN(input_size=env.obs_space,
                       output_size=env.action_space)
    target_model.load_state_dict(model.state_dict())
    optimizer = optim.Adam(model.parameters(), lr=lr)
    memory = deque(maxlen=10000)
    epsilon = 1.0

    rewards = []
```

The initialization phase establishes two neural networks: i) `model`, the primary model that we actively train, and ii) `target_model`, a target model that provides stable Q-value estimates during learning. The target network avoids the situation where we are predicting observations using the same model that we are estimating, since this dual usage of the model can impede convergence. Broadly speaking, reusing the same model for prediction and update is like trying to measure your height with a ruler whose length keeps changing: By freezing the target network parameters for multiple episodes (fixing the length of the ruler, i.e. the “reference frame”), we stabilize the learning process.

The Adam optimizer, called using `optim.Adam(model.parameters(), lr=lr)`, implements adaptive moment estimation, which is a gradient descent variant that maintains separate learning rates for each parameter in the network. Unlike standard stochastic gradient descent which uses a single global learning rate α , Adam computes individual adaptive learning rates from estimates of first and second moments of the gradients. In practice, this means parameters that receive large or frequent gradient updates get smaller effective learning rates, while parameters with small or infrequent updates get larger effective learning rates. This adaptive mechanism often accelerates convergence compared to vanilla

gradient descent, particularly when the loss landscape has different curvatures in different dimensions.

The experience replay buffer stores state transition tuples in a deque with maximum length 10000; observations more than 10000 steps ago will “fall out” the left side of the deque as we append more observations to the right side. This buffer is useful because it breaks the temporal correlations inherent in sequential MDP observations (as the statistician will likely quickly identify, training on consecutive states violates the independence assumptions underlying stochastic gradient descent, leading to inefficient learning and potential divergence). The replay buffer also allows us to reuse samples, so that the model can learn from the same transition multiple times. Reusing samples is particularly valuable for rare but informative transitions such as receiving high rewards or entering terminal states.

The following code block contains an implementation the standard RL interaction cycle between agent and environment. Action selection follows an epsilon-greedy policy where with probability ε the agent selects a random action from the action space $\mathcal{A}$, and with probability $1-\varepsilon$ the agent selects the greedy action $a^* = \arg \max_{a \in \mathcal{A}} Q(x; \theta)$ according to the current Q-network parameters θ . The `torch.no_grad()` context manager disables gradient computation during action selection which reduces unnecessary memory consumption since we only need gradients during the training update phase.

```
for ep in range(episodes):
    state = torch.tensor(env.reset(), dtype=torch.float32)
    total_reward = 0
    done = False

    while not done:
        if random.random() < epsilon:
            action = np.random.randint(env.action_space)
        else:
            with torch.no_grad():
                q_vals = model(state.unsqueeze(0))
                action = torch.argmax(q_vals).item()

        next_state_np, reward, done = env.step(action)
        next_state = torch.tensor(next_state_np, dtype=torch.float32)
        memory.append((state, action, reward, next_state, done))
        state = next_state
        total_reward += reward
```

The state tensor requires unsqueezing to add a batch dimension because neural networks in PyTorch expect inputs with shape `(batch_size, features)` rather than just `(features)`. Even when processing a single state, we must present it as a batch of size one. After executing the selected action, we observe the transition $(x_j, a_j, r_{j+1}, x_{j+1})$ and store it in the replay buffer. This accumulated experience provides the raw material for the learning updates that follow. The loop is continued below.

```

        if len(memory) >= batch_size:
            batch = random.sample(memory, batch_size)
            states, actions, rewards_batch,
            next_states, dones = zip(*batch)

            states = torch.stack(states)
            actions = torch.tensor(actions)
            rewards_batch = torch.tensor(rewards_batch)
            next_states = torch.stack(next_states)
            dones = torch.tensor(dones, dtype=torch.float32)

            q_vals = model(states).gather(1,
            actions.unsqueeze(1)).squeeze()
            with torch.no_grad():
                max_next_q = target_model(next_states).max(dim=1)[0]
            targets = rewards_batch +
            gamma * max_next_q * (1 - dones)

            loss = F.mse_loss(q_vals, targets)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        if ep % 10 == 0:
            target_model.load_state_dict(model.state_dict())

        epsilon = max(epsilon_min, epsilon * epsilon_decay)
        rewards.append(total_reward)

    return rewards

```

For a real-world application, Yauney and Shah used Reinforcement Learning to optimize chemotherapy and clinical trial dosing regimens [186], formulating the chemotherapy dosing problem as a Markov Decision Process as follows, where states X_j represent patient conditions, including tumor size and wellness metrics, actions a_j correspond to different dosage levels of chemotherapy, and rewards r_j are based on patient outcomes, balancing tumor reduction against toxicity effects

Using Q-learning to estimate action-values, they demonstrated that their Reinforcement Learning approach could discover effective individualized treatment regimens directly from clinical data. Their model learned to administer an adaptive sequence of doses that dynamically responded to the patient’s changing condition, increasing doses when necessary to combat tumor growth but reducing them to mitigate toxicity when the patient showed wellness deterioration.

Notably, the optimal treatment strategy typically did not involve constant dosing (as might be prescribed in standard protocols) but rather demonstrated an “up-and-down structure due to its adaptive properties” [186].

At its heart, the Q -function is a type of causal model: Q is effective at deriving the optimal policy to the degree that it accurately represents the causal associations between state and reward. If toggling some component $\tilde{x}$ of the state vector x results in a different value of $\arg \max_{a \in \mathcal{A}} Q(x, a)$, then $\tilde{x}$ is causally associated with the reward R . Our estimate

$\hat{Q}$ of Q , therefore, is useful inasmuch as it correctly captures the causal effect of (X, A) on R . We will return to this principle later, in the discussion of relative ignorability.

5.5 Fitted Q Iteration

So far, all of the Reinforcement Learning Methods we have discussed require *online learning*, that is, we need to be able to curate each sample based on the estimated value or Q-function as fit to the data up until that timepoint: We will not know, for example, what action to assign to the j^{th} observation until we have observed the MDP up to iteration $j - 1$. However, as previously indicated, many real-world applications require learning from pre-collected datasets without further environment interaction. Consider, for example, an autonomous vehicle engineer who must learn driving policies from previously recorded driving data. In these scenarios, we cannot afford to explore new actions that might be harmful or costly; we cannot afford to test the method in real life by applying real actions and seeing if they work or not.

Fitted Q Iteration (FQI) addresses this challenge by learning optimal policies entirely from “offline” data; that is, from a fixed dataset. Unlike online Q-learning, which updates Q-values incrementally as new experiences arrive, FQI treats the entire dataset as a supervised learning problem, fitting a Q-function to satisfy the Bellman equation across all observed transitions simultaneously.

Suppose we have a dataset $\mathcal{D} = \{(x_j, a_j, r_j, x'_j)\}_{j=1}^N$ consisting of N observed transitions, where each tuple represents a state x_j , action a_j , immediate reward r_j , and next state x'_j . This data was collected by some behavior policy π_b , which may be unknown to us. Our goal is to learn a policy π^* that maximizes expected return, using only this fixed dataset.

The idea behind FQI is to reformulate Q-learning as a sequence of supervised learning problems: Rather than updating Q-values one transition at a time, we fit a Q-function Q_k at iteration k using regression targets derived from the previous iteration Q_{k-1} .

Algorithm 6 outlines the FQI procedure. The algorithm begins with an initial Q-function estimate Q_0 and iteratively improves it by solving supervised learning problems.

Algorithm 6 Fitted Q Iteration

Require: Dataset $\mathcal{D} = \{(x_j, a_j, r_j, x'_j)\}_{j=1}^N$, function class $\mathcal{Q}$, convergence threshold ϵ

Ensure: Approximate optimal Q-function Q^*

- 1: Initialize $Q_0 \in \mathcal{Q}$ (e.g., $Q_0(x, a) = 0$ for all $(x, a) \in \Omega \times \mathcal{A}$)
 - 2: Set $k \leftarrow 0$
 - 3: **repeat**
 - 4: Create regression dataset $\mathcal{T}_k = \{((x_j, a_j), \hat{r}_j^{(k)})\}_{j=1}^N$ where
 - 5: $\hat{r}_j^{(k)} = r_j + \gamma \max_{a' \in \mathcal{A}} Q_k(x'_j, a')$
 - 6: Solve supervised learning problem:
 - 7: $Q_{k+1} \leftarrow \arg \min_{q \in \mathcal{Q}} \sum_{j=1}^N (q(x_j, a_j) - \hat{r}_j^{(k)})^2$
 - 8: $k \leftarrow k + 1$
 - 9: **until** $\|Q_k - Q_{k-1}\|_\infty < \epsilon$
 - 10: $Q^* \leftarrow Q_k$
 - 11: **return** Q^*
-

At each iteration k , we construct regression targets $\hat{r}_j^{(k)} = r_j + \gamma \max_{a' \in \mathcal{A}} Q_k(x'_j, a')$ using the current Q-function estimate. These targets represent our best estimate of the true Q-

values based on the Bellman equation. We then fit a new Q-function Q_{k+1} to minimize the squared error between predicted and target values (Equation 5.30).

$$Q_{k+1} \leftarrow \arg \min_{q \in \mathcal{Q}} \sum_{j=1}^N (q(x_j, a_j) - \hat{r}_j^{(k)})^2 \quad (5.30)$$

The perceptive reader should notice that the Q-function here is, in essence, a causal model. This supervised learning problem is identical to the regression framework from Chapter 2: we are estimating $E[\tilde{R}|X = x, A = a]$ where $\tilde{R}$ is our constructed target $\hat{r}_j^{(k)}$. When $\gamma = 0$, FQI reduces to standard treatment effect estimation, simply fitting $Q(x, a) = E[R|X = x, A = a]$ using the observed rewards. The recursive component introduced by $\gamma > 0$ makes FQI a sequence of causal estimation problems, each informed by the previous iteration's estimates of future value.

Consider, for example the case where one wishes to optimize antibiotic treatment protocols from historical medical records. Since the medical records are already generated (treatment is in the past), we obviously cannot choose actions according to an agentic policy. We might therefore use FQI instead. Here, the dataset $\mathcal{D}$ contains patient states x_j (vital signs, lab results), treatments a_j (antibiotic choice and dosage), and outcomes r_j (recovery indicators). The behavior policy π_b represents historical medical practice.

Suppose, however, that this setting involves significant unmeasured confounding: doctors' treatment decisions were influenced by subtle clinical intuitions, patient preferences, and other factors not captured in the electronic health records. These unobserved variables x_m may be critical for determining optimal treatment but are missing from our dataset. But this is just the confounding problem we addressed with MSMs in Chapter 3: treatment assignment depends on unmeasured covariates that also affect outcomes. Naturally, the solution is similar: If we can gather some information about the confounding factors, then we can use techniques like IPTW to augment the FQI process, reweighting observations by importance sampling weights $\frac{\pi^*(a_j|x_j)}{\pi_b(a_j|x_j)}$ to account for the mismatch between the behavior policy and target policy.

If these confounders significantly affect optimal treatment selection then standard FQI will learn biased treatment policies (in this case, we would say that the confounders are *relatively non-ignorable* with respect to the action selection; we will return to relative ignorability in Chapter 7). The algorithm might recommend aggressive treatments for patients who appear healthy in the observed data but who had unmeasured risk factors that led their doctors to choose conservative treatments.

5.6 Temporal Difference learning

In 1988, a different Richard - Richard Sutton - noticed a limitation of value iteration: Algorithm 3 is potentially inefficient because it updates every state after every iteration, even the states that we have not visited [163]. If we assume that treatment effects are different between smokers and non-smokers, does observing a smoker really tell us anything about what would happen to a non-smoker? Not really - and even if it does tell us some marginal amount of information, it may not be worth it to update our estimates of the value function for the non-smokers after observing a smoker if this update comes at a substantial time or resource cost. Perhaps the smokers and the non-smokers share some common features which behave similarly with treatment, but if these features do not tell us

anything about what action to pick, then spending time and statistical power to incorporate them into the model is not efficient.

Instead of updating the value function fully after every episode, the TD update only incorporates changes from states that were actually visited, modifying the value function with finer-grained step sizes. The simplest TD learning algorithm, known as TD(0), updates the value function according to Equation 5.31.

$$V(x_j) \leftarrow V(x_j) + \alpha[R_{j+1} + \gamma V(x_{j+1}) - V(x_j)] \quad (5.31)$$

The term $[R_{j+1} + \gamma V(x_{j+1}) - V(x_j)]$ is called the TD error, representing the difference between the current estimate $V(x_j)$ and the more informed estimate $R_{j+1} + \gamma V(x_{j+1})$. The parameter $\alpha \in (0, 1]$ is the learning rate, which controls the step size of each update (see Figure 5.6).

The TD Error, like the Kalman gain and the negative of the likelihood function, represents how *surprised* we are by the data, assuming our model is correct.

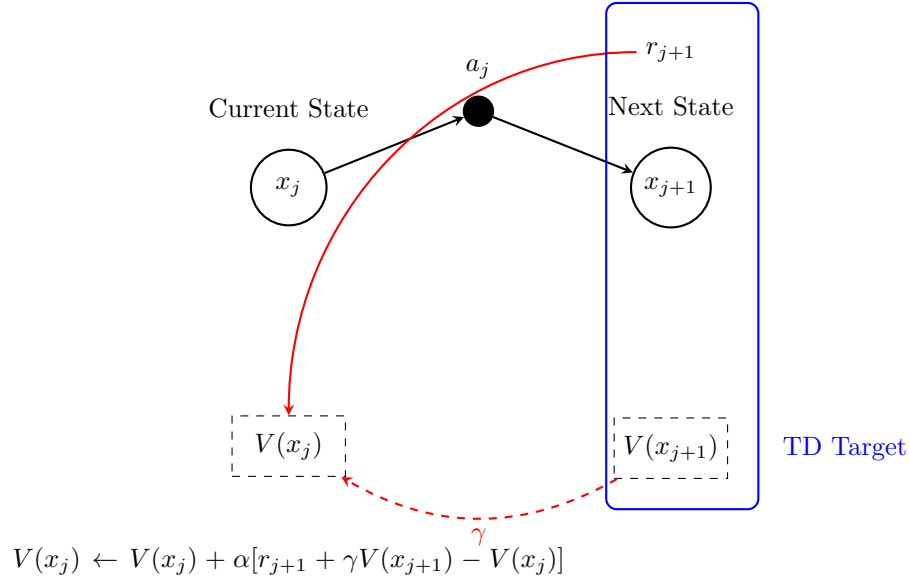


FIGURE 5.6: TD(0) backup diagram. The red arrows show how the value estimate $V(x_j)$ is updated using the immediate reward r_{j+1} and the discounted value estimate of the next state $\gamma V(x_{j+1})$. The blue box highlights the TD target, which serves as the bootstrapped estimate of the true return.

Note that the TD update equation uses a type of “bootstrap” method, updating value estimates based on subsequent estimates rather than waiting for the true return. To illustrate TD(0) learning in practice, we return to the asthma treatment scenario introduced in Chapter 3. Recall that Dr. Yersin treats patients like Marcus who present with varying degrees of respiratory distress, measured by peak flow readings. At each visit j , Dr. Yersin observes Marcus’s current state x_j (his peak flow measurement) and must choose an action $a_j \in \{0, 1\}$, where $a_j = 1$ represents prescribing high-dose steroids and $a_j = 0$ represents conservative management. The reward r_{j+1} is defined as the negative of the number of

asthma exacerbations observed before the next visit, so that higher rewards correspond to better outcomes.

In the MSM framework discussed in Chapter 3, we would collect data from all patients across all timepoints, compute stabilized weights ω_{it} using Equation 5.38, and then fit a regression model to estimate treatment effects. The MSM approach operates in batch mode, processing the entire dataset after collection is complete. In contrast, TD learning updates the value function $V(x_j)$ incrementally after observing each state transition, making it suitable for online learning scenarios where Dr. Yersin must continuously refine her treatment strategy as new patients arrive.

The state space can be discretized based on peak flow ranges. For instance, we might define states as $x \in \{0, 1, 2, 3\}$ corresponding to peak flow ranges $[0, 180)$, $[180, 220)$, $[220, 260)$, and $[260, 500)$ respectively. When Marcus arrives with a peak flow of 185, he is in state $x_j = 1$. The TD(0) algorithm maintains a value estimate $V(x)$ for each state, initialized to zero. As Dr. Yersin treats Marcus and observes the outcome, the algorithm updates $V(x_j)$ according to Equation 5.31.

Suppose, for example, Marcus arrives in state $x_j = 1$ with our current estimate $V(x_j) = -10.5$ (negative because rewards are defined as negative exacerbation counts). Dr. Yersin prescribes steroids, and at the next visit Marcus has improved to state $x_{j+1} = 2$ with peak flow 240, having experienced only one exacerbation, giving reward $r_{j+1} = -1$. Suppose our current value estimate for state 2 is $V(x_{j+1}) = -6.0$. With learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.9$, the TD error is $\delta_j = -1 + 0.9(-6.0) - (-10.5) = -1 - 5.4 + 10.5 = 4.1$. The value function updates to $V(x_j) = -10.5 + 0.1(4.1) = -10.09$. The positive TD error indicates that the outcome was better than expected, leading to an upward revision of the value estimate for state 1.

A minimal implementation of TD(0) for this scenario can be written in Python as follows. The code simulates n patients, each with multiple sequential visits, updating the value function after each observation.

TD-learning

```
#TD_learning.py

import numpy as np
np.random.seed(42)

# Parameters
n_patients = 200
n_visits = 3
state_bins = [0, 180, 220, 260, 500]
n_states = len(state_bins) - 1
V = np.zeros(n_states)

# TD hyperparameters
alpha = 0.1 # Learning rate
gamma = 0.9 # Discount factor

def discretize_state(peak_flow):
    for i in range(len(state_bins) - 1):
        if state_bins[i] <= peak_flow < state_bins[i + 1]:
            return i
    return n_states - 1
```

We use the same treatment policy with confounding that motivated the MSM approach in Chapter 3, where physicians are more likely to prescribe steroids for patients with lower peak flow readings.

```

def simulate_trajectory(patient_id):
    trajectory = []
    peak_flow = np.random.uniform(160, 280)

    for visit in range(n_visits):
        x_j = discretize_state(peak_flow)

        # prob(steroid) increases as peak flow decreases
        prob_steroid = 1.0 / (1.0 + np.exp(0.03 * (peak_flow - 200)))
        a_j = 1 if np.random.rand() < prob_steroid else 0

        # Reward: negative exacerbation count
        baseline = max(0, (250 - peak_flow) / 10)
        treatment_effect = -a_j * (5 + 0.02 * (250 - peak_flow))
        noise = np.random.normal(0, 2)
        exacerbations = max(0, baseline + treatment_effect + noise)
        r_j = -exacerbations

        # State transition
        improvement = a_j * np.random.uniform(10, 30)
        natural_var = np.random.normal(0, 10)
        peak_flow = np.clip(peak_flow + improvement +
                           natural_var, 150, 350)
        x_next = discretize_state(peak_flow)

        trajectory.append((x_j, a_j, r_j, x_next))

        if visit < n_visits - 1 and np.random.rand() < 0.05:
            break

    return trajectory

```

Running TD learning with these specifications produces value estimates that reflect the expected cumulative discounted rewards from each state.

```

# Main TD learning loop
for patient_id in range(n_patients):
    trajectory = simulate_trajectory(patient_id)

    for x_j, a_j, r_j, x_next in trajectory:
        td_error = r_j + gamma * V[x_next] - V[x_j]
        V[x_j] = V[x_j] + alpha * td_error

print("Learned Value Function:")
for i in range(n_states):
    flow_range = f"[{state_bins[i]}, {state_bins[i+1]})"
    print(f"State {i} (peak flow {flow_range}): V = {V[i]:.3f}")

```

A typical output shows $V(x = 0) \approx -13.4$, $V(x = 1) \approx -10.1$, $V(x = 2) \approx -7.7$, and $V(x = 3) \approx -6.9$, indicating that patients with higher peak flows (better respiratory

function) have better expected outcomes. The monotonic relationship between peak flow and value estimates confirms that the algorithm has learned the underlying structure of the problem.

TD learning and MSMs primarily differ in when and how the updates occur. MSMs compute weights for all observations (Equation 5.38), then fit a single weighted outcome regression model to estimate the treatment-effect parameters β . This batch approach requires storing all data and calculating propensity scores $\pi(a_{it'}|x_{it'})$ for every timepoint. In contrast, TD learning updates incrementally, requiring only the current state x_j , action a_j , reward r_{j+1} , and next state x_{j+1} ; as a result, TD learning is fairly memory-efficient and more suitable than MSMs for online applications where treatment policies must be refined continuously as new data arrives.

But at the end of the day, TD-learning and MSMs are different strategies for dealing with time-dependent confounding in sequential decision problems. The stabilized weights in MSMs (Equation 5.38) reweight observations to break the confounding paths $A_{j-1} \rightarrow X_j \rightarrow A_j$. The TD update, on the other hand, implicitly accounts for the confounding structure through bootstrapping: by updating $V(x_j)$ based on the observed reward and the value of the subsequent state $V(x_{j+1})$, the algorithm learns to predict long-term outcomes while accounting for the fact that treatment decisions at time j affect both immediate rewards and future states; the structure is learned empirically by observing when (at what timepoint) and for whom (with which covariates) different strategies are optimal. When importance sampling ratios are incorporated into TD updates as in Equation 5.40, the connection becomes even more explicit, with the importance weights serving the same function as the MSM stabilized weights.

The convergence properties of TD learning differ from those of MSMs in important ways. Under standard regularity conditions, MSM estimators are consistent and asymptotically normal as the number of subjects $n \rightarrow \infty$. TD learning, however, converges as the number of state visits increases, which may require multiple passes through the data or a large number of trajectories. The learning rate α controls the speed and stability of convergence: larger values of α lead to faster initial learning but can cause instability, while smaller values ensure convergence but require more data. In practice, researchers often use a decaying learning rate $\alpha_j = \alpha_0/(1 + j/\tau)$ where τ controls the decay rate, ensuring that early observations have strong influence while later updates make fine-grained adjustments.

For the asthma application, the choice between MSMs and TD learning depends on the research context. If the goal is to estimate the average treatment effect of steroids across the population, or to perform a one-time analysis of a completed observational study, MSMs offer a natural framework with well-established inferential properties. If instead Dr. Yersin wishes to develop an adaptive decision support system that improves continuously as she treats new patients, TD learning provides a principled online learning mechanism. The connection between these approaches, formalized through the equivalence of stabilized weights and importance sampling ratios, ensures that insights from one framework can often be translated to the other.

The type of TD learning we have discussed so far is actually a special case called TD(0). Here, our TD(0) updates are based solely on the reward and value estimate from the next state. However, this need not be so: we can extend this idea to incorporate information from multiple future steps to increase the robustness of our predictive system. The n -step TD update uses the n -step return, defined in Equation 5.32.

$$G_j^{(n)} = R_{j+1} + \gamma R_{j+2} + \gamma^2 R_{j+3} + \cdots + \gamma^{n-1} R_{j+n} + \gamma^n V(x_{j+n}) \quad (5.32)$$

The corresponding update rule is shown in 5.33.

$$V(x_j) \leftarrow V(x_j) + \alpha[G_j^{(n)} - V(x_j)] \quad (5.33)$$

As n increases, the n -step return approaches the full return used in Monte Carlo methods. When $n = 1$, we recover TD(0). This spectrum illustrates the continuum between TD(0) and Monte Carlo methods, with the former representing one extreme (looking ahead one step) and the latter representing the other (looking ahead to the episode's end). The n -step TD update is pictured in Figure 5.7.

Note that the usage of n here differs from the conventional usage of n in statistical methods. The n in n -step TD learning has nothing to do with sample size.

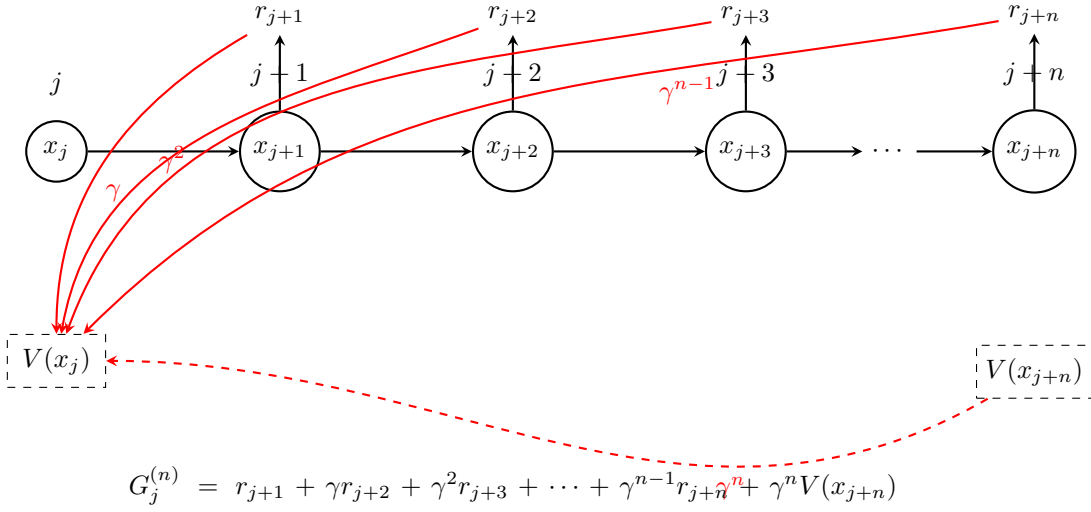


FIGURE 5.7: n -step TD backup diagram. The value estimate $V(x_j)$ is updated using rewards from the next n steps and the bootstrapped value estimate $V(x_{j+n})$. Each reward is discounted by the appropriate power of γ .

Figure 5.7 leads to the obvious question of how many steps to include for the return: What is the optimal value of n ? The selection of n is itself a mini-bandit problem within the Reinforcement Learning framework, where there exists a set of parallel universes (potential outcomes), each of which corresponds to us selecting a different value of n . Armed with our statistical models of V and Q , we can consider them all instead of just one, by computing a weighted average of n -step returns for all possible values of n . This approach is called TD(λ).

TD(λ) implements the multi- n idea using a concept called the *eligibility trace*, wherein one records which states have been recently visited and only considers those recently-visited states as eligible for update. The parameter $\lambda \in [0, 1]$ controls how quickly the eligibility of past states decays.

The λ -return is defined in Equation 5.34, which represents a weighted average of all n -step returns, with weights proportional to λ^{n-1} .

$$G_j^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_j^{(n)} \quad (5.34)$$

With the return G_j^λ in hand, the $\text{TD}(\lambda)$ update is subsequently defined in Equation 5.35.

$$V(x_j) \leftarrow V(x_j) + \alpha[G_j^\lambda - V(x_j)] \quad (5.35)$$

In practice, $\text{TD}(\lambda)$ is often implemented using eligibility traces, which track the states that have been visited and are eligible for update. The traces are updated according to Equation 5.36, and the value function is updated for all states per Equation 5.37, where $\delta_j = R_{j+1} + \gamma V(x_{j+1}) - V(x_j)$ is the TD error.

$$e_j(x) = \begin{cases} \gamma \lambda e_{j-1}(x) & \text{if } x \neq x_j \\ \gamma \lambda e_{j-1}(x) + 1 & \text{if } x = x_j \end{cases} \quad (5.36)$$

$$V(x) \leftarrow V(x) + \alpha \delta_j e_j(x) \quad \text{for all } x \in \Omega \quad (5.37)$$

The TD-family of methods in Reinforcement Learning are related to Marginal Structural Models (MSMs) from Causal Inference. Consider a subject i at timepoint t in an episodic process, with action a_{it} , covariates x_{it} , and reward $r_{i,t+1} = \rho(x_{i,t+1})$. Then, the stabilized weights at time t are defined in Equation 5.38, wherein $\bar{a}_{i(t'-1)}$ represents the action history through time $t' - 1$ and $\bar{x}_{it'}$ represents the covariate history through time t' .

$$\omega_{it} = \prod_{t'=0}^t \frac{\pi(A_{it'} = a_{it'} | \bar{a}_{i(t'-1)})}{\pi(A_{it'} = a_{it'} | \bar{a}_{i(t'-1)}, \bar{x}_{it'})} \quad (5.38)$$

These stabilized weights are nearly identical to the importance sampling ratios used in TD-learning. At each time step, we compute the ratio between i) The probability of the observed action under a simplified model that depends only on past actions ($\pi(A_{it'} = a_{it'} | \bar{a}_{i(t'-1)})$), and ii) The probability of the observed action under a more complex model that depends on both past actions and covariates ($\pi(A_{it'} = a_{it'} | \bar{a}_{i(t'-1)}, \bar{x}_{it'})$).

The cumulative product of these ratios (Equation 5.38) creates a weighting system that effectively transforms the observed data distribution into a different distribution where action assignment depends only on past actions, thereby emulating the effect of randomization.

$$\hat{V}^\pi(x_j) = \frac{\sum_{j'=1}^J \prod_{k=j'}^{T-1} \frac{\pi(a_k | X_k)}{\pi_b(a_k | X_k)} G_{j'} \mathbb{I}(X_{j'} = x_j)}{\sum_{j'=1}^J \mathbb{I}(X_{j'} = x_j)} \quad (5.39)$$

Compare Equation 5.38 to the importance sampling estimator in Reinforcement Learning from Section 4.5, which is reproduced for convenience in Equation 5.39: Note that in both equations, we use products of probability ratios to reweight observed sequences, thereby accounting for the full history of actions and state transitions. The most substantive difference is that the MSM version uses two indexing variables, i, t while the more general importance sampling weights use j : MSMs use a “wide” format of the data as in Table 5.3, while importance sampling ratios use the “long” version as in Table 5.4 -that’s why we need two indices i, t on the lefthand side of Equation 5.38, but only one index j on the lefthand side of Equation 5.39.

Now, let us consider the eligibility trace update in off-policy TD- λ (Equation 5.36) with the value function updated according to Equation 5.37, where $\delta_j = r_{j+1} + \gamma V(x_{j+1}) - V(x_j)$ is the TD error.

TABLE 5.3: Sequential treatment data in wide format. Each row represents a patient with treatment sequences and outcomes at different timepoints.

	Patient	Treatment 1 $t = 1$	Treatment 2 $t = 2$	Outcome 1 $t = 1$	Outcome 2 $t = 2$
$i = 1$	Claudius	A	B	3	4
$i = 2$	Julius	A	A	6	9
$i = 3$	Augustus	B	A	5	7
$i = 4$	Octavian	B	B	10	2

TABLE 5.4: Sequential treatment data in long format. Each row represents a single observation at a specific timepoint for a patient.

	Patient	Treatment	Outcome	Timepoint
$j = 1$	Claudius	A	3	Day 5
$j = 2$	Claudius	B	4	Day 10
$j = 3$	Octavian	B	10	Day 5
$j = 4$	Octavian	B	2	Day 10
$j = 5$	Julius	A	6	Day 5

For the off-policy case, the update (Equation 5.40) includes the same importance sampling ratios as before, where $\frac{\pi_t(a_j|X_j)}{\pi_b(a_j|X_j)}$ is the one-step importance sampling ratio between target policy π_t and behavior policy π_b . This recursive update creates a weighted average of returns across different time scales, with the eligibility trace maintaining a decaying memory of state visitations.

$$V(x) \leftarrow V(x) + \alpha \delta_j \frac{\pi_t(a_j|X_j)}{\pi_b(a_j|X_j)} e_j(x) \quad (5.40)$$

Similarly, MSMs for time-varying actions must account for the accumulating effect of action decisions through time, with stabilized weights capturing the sequential nature of action assignment; like the TD methods, MSMs also propagate weights recursively through a sequence of observations, adjusting for the mismatch between observed distributions and target distributions of interest. Said another way, the TD update equation with eligibility traces is an iterative reweighting scheme analogous to the sequential weighting approach in MSMs. Precise determination of when the estimators and weights are equivalent between the two methods is a compelling direction for future research.

5.7 Policy Gradient Methods

Most Reinforcement Learning methods we have discussed so far involve the estimation of a value function (either V or Q), and then subsequent derivation of the policy as the argmax over the action-value function. This approach is called *value-based* reinforcement learning. An alternative approach is to parameterize the policy directly as $\pi_\theta(a | x)$ and optimize θ to maximize expected return, without ever explicitly representing Q . This approach is called *policy gradient*, and it forms the second major family of modern RL methods. To keep this chapter at a reasonable length, we will only discuss the mathematical details which are necessary to understand the connection to Reinforcement Learning; readers can

refer to Chapter 13 of [164] for a thorough introduction. Classical policy gradient methods are inherently *on-policy*: the gradient estimator is unbiased only when the data being used for the update were generated by the current policy π_θ . Later in this section, we will discuss the *off-policy* case (which, like neural fitted Q-iteration, requires importance-weighting methods).

Direct policy optimization is most useful when the decision policy is simple, and the action space is very complex or high-dimensional. Value-based methods require an arg max over the action space at each decision point, which can be intractable in the complex action-space case: Consider, for example, a robot controlling a 12-degree-of-freedom manipulator. The robot cannot feasibly enumerate all possible joint torques to select the best one. But if we can parameterize the policy directly in a more compact representation than the explicit state parameterization, then the robot can optimize its actions directly without requiring a value function.

Policy gradient methods also share a mathematical correspondence with the adjoint method from optimal control theory (Section 5.4). In lieu of Bellman’s dynamic programming solution [13], policy gradient methods instead approximate Pontryagin’s maximum principle [123, 87], creating a reunion of two traditions which had evolved separately since the 1960s [119]. In brief, the deterministic policy gradient [157] can be derived as a special case of policy gradients: In this special case, the optimal policy is obtained *via* backpropagation through the trajectory of state-action-reward tuples [59, 147]. And the reader should recall that backpropagation is itself the adjoint method of classical optimal control [27] applied to a parameterized policy [35, 181].

The simplest policy gradient algorithm is REINFORCE, developed by Williams [183], which estimates $Q^{\pi_\theta}(x_j, a_j)$ by the Monte Carlo return G_j and averages the resulting gradient estimator over sampled trajectories. Algorithm 7, from [183] and [164], states the procedure formally.

Algorithm 7 REINFORCE (Monte Carlo Policy Gradient)

Require: Differentiable policy parameterization $\pi_\theta(a \mid x)$, step size $\alpha > 0$, discount factor $\gamma \in [0, 1]$

Ensure: Optimized policy parameters θ

```

1: Initialize policy parameter  $\theta$  (e.g., randomly or with small weights)
2: repeat
3:   Generate an episode  $x_0, a_0, r_1, x_1, a_1, r_2, \dots, x_{T-1}, a_{T-1}, r_T$  following  $\pi_\theta$ 
4:   for  $j = 0$  to  $T - 1$  do
5:      $G \leftarrow \sum_{k=j+1}^T \gamma^{k-j-1} r_k$  ▷ Return from step  $j$ 
6:      $\theta \leftarrow \theta + \alpha \gamma^j G \nabla_\theta \log \pi_\theta(a_j \mid x_j)$ 
7:   end for
8: until stopping criterion is met
9: return  $\theta$ 

```

One practical issue with REINFORCE is that the Monte Carlo return G_j has variance that grows with trajectory length; in addition, REINFORCE must wait until the end of an episode to compute the return, which is impractical for long or continuing tasks. The *actor-critic* family of methods has been developed in response to this issue, introduced in their modern form by Konda and Tsitsiklis [86]. Actor-critic methods mitigate both of these issues by creating a second function approximator and optimizing both simultaneously. This second function is called a *critic*. Our original function, which we call the *actor* parameterizes the policy $\pi_\theta(a \mid x)$ and makes choices, while the *critic* estimates the value function ($V^{\pi_\theta}(x; \phi)$ or $Q^{\pi_\theta}(x, a; \phi)$, where ϕ denotes the critic’s parameters). In the training loop,

the critic is updated first via temporal difference learning (Section 5.6), and the actor is updated using the critic’s value estimates in place of Monte Carlo returns. This TD-error estimator has lower variance than the Monte Carlo return (leading to more stable learning), because the bootstrapped one-step target depends only on a single transition, whereas the Monte Carlo return accumulates noise from every step of the trajectory.

Most modern actor-critic implementations use a slightly different empirical TD-error than the one described in section 5.6. Our new TD error, called the *advantage score* or *advantage* of action a at state x , is derived to estimate $A(x, a) := Q(x, a) - V(x)$; the advantage score measures how well an action is estimated to perform *relative to the average* over actions in that state. In the following section, we will examine this decomposition formally and see its connection to Causal Inference (in fact, the advantage score is a recalibrated blip function). Actor-critic methods combine the low bias of policy gradient with the low variance of bootstrapped value estimates; for this reason, the actor-critic family appears commonly in modern RL research. A3C [107], for example, is a method which parallelizes actor-critic across multiple workers; DDPG [96] extends it to continuous action spaces using the deterministic policy gradient theorem [157].

The Advantage learning (A-learning) strategy is a modern Reinforcement Learning approach which is foundational on the concept of relative action value. In A-learning, we decompose the Q-function into two components: i) the state value $V(x)$, representing how good it is to be in state x under the current policy, and ii) the advantage $A(x, a)$, representing how much better action a is compared to the average action in that state. In equation 5.41, the term $A_\pi(x, a) = Q_\pi(x, a) - V_\pi(x)$ is the *advantage* of choosing a in state x compared to the average action, and $V_\pi(x) = E_{a \sim \pi}[Q_\pi(x, a)]$ is the expected value under policy π .

$$Q_\pi(x, a) = V_\pi(x) + A_\pi(x, a) \quad (5.41)$$

A note on terminology: the term “A-learning” in the biostatistics literature conventionally refers to Robins’ G-estimation method for structural nested mean models (see [132, 113]), in which one models the contrast (blip) function $\psi(a, x)$ directly rather than the full conditional mean. The advantage decomposition described in this section is the Reinforcement Learning analogue of A-learning in that sense; the formal connection between the two is established in Theorem 5.2 later in this section, which is why I use both terms for the same family of methods.

Readers familiar with the RL literature may also be aware that “Advantage Learning” is sometimes used to refer specifically to Baird’s algorithm [9], a value-update method that rescales the action-gap by a factor of $1/\Delta t$ to improve function approximation in fine-discretization settings. Baird’s algorithm is distinct from the advantage decomposition $Q = V + A$ discussed here, which is a more general mathematical object that underlies a wide family of methods including actor-critic, dueling DQN, and advantage-weighted regression. Throughout this book, “A-learning” and “Advantage Learning” refer to the decomposition, not to Baird’s specific algorithm.

Note that relative-value contrasts (as in the blip models discussed below), are only identifiable up to some arbitrary reference, either a baseline reference action or a null value. Usually researchers will require that advantages average to zero under the policy, as described above; the decomposition $Q_\pi = V_\pi + A_\pi$ achieves this automatically, since defining $A_\pi = Q_\pi - V_\pi$ with $V_\pi(x) = E_{a \sim \pi}[Q_\pi(x, a)]$ guarantees that $E_{a \sim \pi}[A_\pi(x, a)] = 0$ for all states x . This choice of reference frame ensures that $V(x)$ truly represents the expected value, while $A_\pi(x, a)$ captures only the relative merit of specific actions compared to the average of all of the actions. Note, however, that the choice of reference frame is arbitrary;

one might just as easily require that $A_\pi(x, a^{(1)}) = 0$, thereby defining some action $a^{(1)}$ as the default or “null” action.

Consider a simplified ICU scenario with three actions available at each decision point:

1. $a^{(1)}$: Maintain current settings
2. $a^{(2)}$: Increase ventilator pressure support
3. $a^{(3)}$: Administer bronchodilators

Suppose at state x (Marcus with 88% oxygen saturation), the true Q-values are:

$$Q(x, a^{(1)}) = 85 \quad (5.42)$$

$$Q(x, a^{(2)}) = 92 \quad (5.43)$$

$$Q(x, a^{(3)}) = 91 \quad (5.44)$$

Then, the state value is $V(x) = \frac{1}{3}(85 + 92 + 91) = 89.33$ (assuming uniform random policy for illustration). The advantages can be computed as follows:

$$A(x, a^{(1)}) = 85 - 89.33 = -4.33 \quad (5.45)$$

$$A(x, a^{(2)}) = 92 - 89.33 = +2.67 \quad (5.46)$$

$$A(x, a^{(3)}) = 91 - 89.33 = +1.67 \quad (5.47)$$

Based on these advantages, we can conclude that maintaining current settings is harmful relative to taking action, while increasing pressure support provides the greatest benefit.

Decomposing the Q function as such, into value + advantage, also improves the model’s ability to generalize. Many function approximators struggle to represent the large dynamic range of Q-values across different states, but by factoring out the state-dependent component $V(x)$, the advantage function $A(x, a)$ often has a smaller range and simpler structure, making it easier to approximate with neural networks or other function approximators.

In Reinforcement Learning, the advantage serves the same purpose as the blip function does in the structural nested mean model (SNMM) framework underlying the g-estimation strategy of section 3.5. The only substantive difference is that the blip function typically compares one treatment to another treatment, while the advantage function compares one treatment to the average of the treatments. To see this, consider Equation 5.48, which shows the advantage under a policy π which selects actions uniformly with probability $1/k$.

$$A(x, a) = Q(x, a) - \frac{1}{k} \sum_{a' \in \mathcal{A}} Q(x, a') \quad (5.48)$$

If we define the blip function relative to the average treatment effect rather than a null treatment, then Equations 5.49-5.51 arise, from which we see that the blip and advantage functions can be identical.

$$\psi(a, x; \beta) = E[R|X = x, A = a] - \frac{1}{k} \sum_{a' \in \mathcal{A}} E[R|X = x, A = a'] \quad (5.49)$$

$$= Q(x, a) - \frac{1}{k} \sum_{a' \in \mathcal{A}} Q(x, a') \text{ (by def. of Q, assuming uniform policy)} \quad (5.50)$$

$$= A(x, a) \text{ (by 5.48)} \quad (5.51)$$

The Advantage-Blip Equivalency is stated formally in 5.2.

Theorem 5.2 (Blip-Advantage Equivalence) *Let $\mathcal{A}$ be a finite action set with $|\mathcal{A}| = k$. Define the blip function as $\psi(a, x; \beta) = \mathbb{E}[R|X = x, A = a] - \frac{1}{k} \sum_{a' \in \mathcal{A}} \mathbb{E}[R|X = x, A = a']$ (i.e., centered relative to the average treatment effect). Define the advantage function under a uniform policy $\pi(a|x) = 1/k$ as $A^\pi(x, a) = Q^\pi(x, a) - V^\pi(x)$. Then for all x and a ,*

$$\psi(a, x; \beta) = A^\pi(x, a). \quad (5.52)$$

Proof. Under the uniform policy, $V^\pi(x) = \sum_{a'} \pi(a'|x) Q^\pi(x, a') = \frac{1}{k} \sum_{a'} Q^\pi(x, a')$. Since the single-step Q-function satisfies $Q^\pi(x, a) = \mathbb{E}[R|X = x, A = a]$, the advantage is

$$A^\pi(x, a) = Q^\pi(x, a) - V^\pi(x) = \mathbb{E}[R|X = x, A = a] - \frac{1}{k} \sum_{a'} \mathbb{E}[R|X = x, A = a'] = \psi(a, x; \beta). \quad \square$$

Note that there are four conditions that must be in place in order for the advantage-blip equivalence to hold.

1. **Uniform policy.** If the policy is non-uniform, then $V^\pi(x) = \sum_{a'} \pi(a'|x) Q(x, a')$ is a weighted average rather than a simple average, and $A^\pi(x, a) \neq \psi(a, x; \beta)$ unless ψ is also redefined with the same weights. Under a non-uniform policy the blip function becomes a *weighted* advantage, which still preserves the action ranking but changes the magnitudes.
2. **Finite, discrete actions.** With continuous actions, the sum $\frac{1}{k} \sum_{a'}$ must be replaced by an integral $\int_{\mathcal{A}} \pi(a'|x) da'$, and the centering conventions in the CI and RL literatures may differ.
3. **Centering relative to the average.** The standard blip function in Robins' structural nested mean model [132] is defined relative to a reference treatment $a^{(0)}$: $\psi(a, x; \beta) = \mathbb{E}[R|X = x, A = a] - \mathbb{E}[R|X = x, A = a^{(0)}]$. Note that this pairwise contrast is *not* the same as the advantage function, which centers relative to the policy-weighted average; the two coincide only when centering is performed *symmetrically over all actions* as in Equations 5.49–5.51.
4. **Conditional exchangeability.** Robins' blip is a *counterfactual* contrast: Ψ is defined in terms of the potential outcomes $R \mid a$, not the observed conditional means. Theorem 5.2 defines ψ directly through $\mathbb{E}[R \mid X = x, A = a]$, so the identity holds as stated. But for either side to equal the causal blip of the structural nested mean model, the conditional means must identify the counterfactual contrasts, which requires conditional exchangeability given X (along with the consistency and positivity conditions of Chapter 3). Without exchangeability, the theorem still links the advantage to a regression contrast, but that contrast no longer carries the causal interpretation that makes it a blip function.

Conventional RL notation uses $A_\pi(x, a)$ for the advantage score as I have done here. Since we have now established that the blip and advantage functions serve the same goal, I will henceforth use ψ for the advantage/blip function to avoid confusion with the action A , and to maintain consistency with the ψ used to denote blip functions.

Another difficulty with vanilla policy gradient is that large parameter updates can produce policies very different from the policy that generated the training data, which violates the on-policy assumption that the gradient estimator is based on. Trust Region Policy Optimization (TRPO) [148] addresses this by constraining each update to a KL-divergence ball around the current policy, and Proximal Policy Optimization (PPO) [149] is a first-order

approximation to TRPO that is widely used in modern deep RL. Both methods rely on the importance ratio $\omega_j(\theta) = \pi_\theta(a_j | x_j) / \pi_{\theta_{\text{old}}}(a_j | x_j)$ between the current and previous policy to reweight the gradient estimator. Recall that the importance ratio corrects for the mismatch between the policy that generated the data ($\pi_{\theta_{\text{old}}}$) and the policy being evaluated (π_θ), serving the same role in RL that inverse probability weighting serves in marginal structural models for sequential treatments.

Conventional RL notation differs from the notation used in this section in two ways. First, the RL literature typically denotes a trajectory by $\tau = (x_0, a_0, x_1, a_1, \dots, x_T)$, so the policy gradient objective is often written $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G_0]$ rather than $J(\theta) = \mathbb{E}_{\pi_\theta}[G_0]$. We avoid τ here because the symbol is used elsewhere in this book for treatment effects and other quantities. Second, the PPO and TRPO literature [149, 148] denotes the importance ratio as $r_j(\theta)$. We use $\omega_j(\theta)$ throughout to maintain consistency with Section 4.5 and to avoid collision with the specific reward r .

Causal Inference also has a set of methods developed for learning the optimal policy directly; this type of method, developed in the field of econometrics, is referred to as *Optimal Policy Learning* (OPL). OPL was pioneered by Manski [103], who posed the econometric problem in its modern form: *Given observational data on heterogeneous observations, what is the treatment assignment rule that maximizes total welfare?* This seminal work provides methodology to directly optimize an econometric decision policy, under the identifiability conditions from Chapter 3 (exchangeability, positivity, consistency). In fact, the usage of the word, “policy,” here, is a beautiful *double-entendre*: The political policy is the RL policy being optimized. Then, in a similar vein as advantage learning, Kitagawa and Tetenov [83] derived *regret* bounds for these policies, where “regret,” in their usage, is the welfare gap between the rule you learned and the rule you would have learned if you had infinite data⁴. Zhao et al. [190] subsequently recast the problem as weighted classification with IPTW weights, and Athey and Wager [7] combined doubly robust scores with the policy search.

OPL, A-learning, and the Q-learning approach from Chapter 4 converge on the same π^* when their modeling assumptions hold; however, each parameterization poses the estimation problem slightly differently. Q-learning (and Cerulli’s regression-based generalized first-best rule) puts the modeling effort into the full conditional mean $\mu(a, x) = \mathbb{E}[R | A = a, X = x]$ and then takes the argmax, with the accuracy of the between-action contrasts contingent on the accuracy of the action-value estimates. In the Manski/Athey-Wager lineage, OPL targets the *policy itself* by constructing a doubly robust value estimator $\hat{V}(\pi)$ and then optimizing over a constrained policy class Π (see Table 5.5); A-learning instead parameterizes the *contrast* $C(a, x) = \mu(a, x) - \mu(0, x)$ explicitly (note that OPL, classic policy gradient methods, and A-learning all leave the action-independent baseline unmodeled).

Note that each column of Table 5.5 differs in *which* mathematical object gets modeled, not *how complex* that model has to be: Any column can use a simple or a complex function class for estimation depending on the application. The trilemma is about *parameterization*; the bias-variance tradeoff within each column is a separate question. Convergence rates therefore depend on the function class chosen for estimation. Zhao et al.’s outcome-weighted learning [190], for example, achieves a rate of $n^{-2/(d+2)}$ because it uses a rich kernel-based policy class; Athey and Wager’s policy search over depth- k trees [7] achieves $O(\sqrt{\text{VC}(\Pi)/n})$ regret with no dependence on the ambient dimension. Q-learning and A-learning face the same tradeoff: parametric models give $\sqrt{n}$ rates, while nonparametric models pay the curse. The only genuine asymmetry in efficiency can be reduced to the difference between discrete

⁴an RL researcher might call the same quantity “suboptimality”

TABLE 5.5: Three policy-learning parameterizations of the same estimand π^* .

	Q-learning / gfb		A-learning		Policy-search OPL
Modeling target	Conditional $\mu(a, x)$	mean	Contrast $\mu(0, x)$	$\mu(a, x) - \mu(0, x)$	Policy $\pi \in \Pi$
Derived policy	$\arg \max_a \hat{\mu}(a, x)$		$\arg \max_a \hat{C}(a, x)$		$\arg \max_{\pi \in \Pi} \hat{V}(\pi)$
Robustness	Single-robust in μ		Doubly robust for C		Doubly robust for $V(\pi)$
Origins	Watkins (RL); G-computation (CI)		Robins; Murphy		Manski; Athey-Wager
RL analog	Q-function methods		Advantage methods		Classic policy gradient

and continuous data: If the data are well-separated by the decision threshold, then sorting items into a category (which action to apply) can be easier than creating accurate forecasts for that item, and therefore the former will require less data.

5.8 Further Reading

Bellman's original monograph [13] provides the mathematical foundations for dynamic programming, and his later memoir *Eye of the Hurricane* [14] offers fascinating historical context about the development of these ideas. For a comprehensive modern treatment of Markov decision processes, see Puterman's text [125].

Watkins' thesis [178] introduced Q-learning and remains surprisingly readable to the amateur. Sutton's seminal paper on TD learning [163] laid the groundwork for temporal difference methods. For those interested in the convergence proofs, see Jaakkola et al. [70] and Tsitsiklis and Van Roy [169].

The deep learning revolution in Reinforcement Learning began with Mnih et al.'s work on Deep Q-Networks [108], which successfully combined neural networks with Q-learning to play Atari games. For those interested in how neural network approximations $Q(x, a; \theta)$ extend our tabular framework, see van Hasselt et al. [171] on Double Q-learning and Hessel et al. [64] on Rainbow, which combines multiple improvements.

The connections between Reinforcement Learning and optimal control theory are explored in depth by Bertsekas [17]. For readers interested in how our value function $V(x)$ relates to the Hamilton-Jacobi-Bellman equation, see Lewis et al. [95].

The application of Reinforcement Learning to medical decision-making is covered extensively in Yu et al.'s survey [187]. Yauney and Shah's work on chemotherapy dosing [186] exemplifies how these theoretical frameworks translate to practical clinical applications.

For a comprehensive modern treatment that unifies many of the concepts in this chapter using similar notation to ours, Sutton and Barto's second edition [164] remains the standard reference. The treatment of eligibility traces in TD(λ) and their connection to TD(0) is particularly well-explained.